

Explicit Visual World Models For Robotic Reasoning



Bruno Hawkins

Supervisor: Dr. Ayush Tewari, Mr. Felix O'Mahony

Department of Engineering
University of Cambridge

This dissertation is submitted for the degree of
MPhil in Machine Learning and Machine Intelligence

Emmanuel College

August 2026

Declaration

I, Bruno Hawkins of Emmanuel College, being a candidate for the MPhil in Machine Learning and Machine Intelligence, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.

Signed Bruno Hawkins

Date 11 August 2026

Statement of software and resources used

This project builds on the VDAWorld codebase of O’Mahony et al. (VLM-directed simulator synthesis, execution, and repair). I extended it for action-conditioned world-model generation, validation, CEM/MPC control, and evaluation. Unmodified inherited infrastructure retained for understanding is included in the repository and is not claimed as original. My extensions and all generated simulator programmes evaluated in this report are my own work.

Software and resources used:

- VDAWorld (inherited base; extended as above) — Chapters 3–5.
- LeWM published code and frozen checkpoints (quentinll/lewm-*). Stable-WorldModel v0.1.1 — Chapter 4 baseline comparison.
- Benchmark environments/datasets: Two-Room, Reacher, Push-T, OGBench-Cube (LeWM evaluation suite) — Chapters 4–5.
- Google Gemini 3.6 Flash (generation VLM) and Gemini 2.5 Flash (segmentation tools). No training of these models — Chapters 3–5.
- Isambard-AI (NVIDIA GH200) for experiments — Chapters 4–5.

- Claude Code (Anthropic) for discussing ideas, making code suggestions and debugging.

This project used only publicly available simulation benchmarks and datasets. No human participants were involved, and no Department Research Ethics Committee approval was required.

No work undertaken before starting the MPhil was reused in this project.

The code and task-agnostic prompt are available at: <https://gitlab.developers.cam.ac.uk/belh2/MLMI-Project-belh2>.

Word count: 14,867 words (excluding front matter, bibliography and figures, and including tables, footnotes, captions and appendices, in accordance with Section 8.1 of the MLMI Course Handbook).

Bruno Hawkins
August 2026

Abstract

To plan a movement before making it, a robot needs a *world model*: an internal representation that predicts how a scene will change if an action is taken. The strongest visual world models predict in a learned latent space. They plan well, but what they represent and how they update it are hidden inside a neural network, so when they fail the cause can not be seen clearly.

This dissertation asks whether a Vision–Language Model (VLM) can write that world model as Python code instead, making the state, dynamics, and objective visible and their errors locatable. The system is given eight demonstrations of a task, each a sequence of camera frames paired with the actions that produced them. From this, the VLM writes code that extracts the scene state from an image, predicts the next state from an action, and scores how far a predicted state is from the goal, self-correcting against internal validation checks. At test time, a standard sampling-based planner queries this code, replanning as the robot moves. The baseline for comparison is LeWM, a recent latent world model evaluated on the same four benchmark environments, trained on roughly ten thousand episodes per environment rather than eight demonstrations. It was rerun locally so both systems attempted the same 50 cases per benchmark under the same success criteria.

Twenty programmes were generated per benchmark, 80 in total, across six mixes of demonstration trajectory types. Where the deciding physics is clearly visible in a 2D task, the generated code world model won: 50/50 against LeWM’s 43/50 navigating a room divided by a wall with one doorway, and 46/50 against 27/50 driving a two-joint arm. Where contact dynamics or 3D space were factors, it lost: 10/50 against 47/50 pushing a T-shaped block into position, and 12/50 against 22/50 moving a cube. Repeating runs on identical configurations also produced large variance in results, so a single generation is not a reliable sample of the method. Since the model is code, every failure was explicit. Nineteen of twenty T-pushing programs left the pusher out of their scoring function, so plans abandoned the pusher once the block was placed. No programme represented height, so none lifted the cube off the table. The internal self-checks caught neither fault, since a programme can not be found inconsistent about a quantity it never represents.

This is the first system to generate an executable, continuous-action world model and its planning objective from image–action trajectories and evaluate it via closed-loop planning on benchmarks standard for latent world modelling systems. Contact-dynamics and 3D tasks remain unsolved. However, every failure was identifiable in a few lines of readable code, which is what a latent model can not offer.

Dedicated to my family, friends, and everyone who has supported me along the way. A special thank you to my mother and father for everything you have done to help me reach this milestone in my academic career and life.

Acknowledgements

I am grateful to my supervisor, Dr Ayush Tewari, for guidance throughout this project. I thank Felix O'Mahony for the VDAWorld codebase and for discussions that helped me extend his work to action-conditioned world models for control. Also, thank you for looking over my drafts and making suggestions to help me improve the content.

Table of contents

List of figures	xv
List of tables	xvii
1 Introduction	1
1.1 Problem Statement	1
1.2 Motivation	2
1.3 Proposed Approach	3
1.4 Research Questions and Contributions	3
1.5 Dissertation Structure	5
2 Background and Related Work	7
2.1 World Models as Internal Predictive Models	7
2.2 Model-Based Planning and Model Error	7
2.3 Latent Visual World Models and JEPAs	9
2.4 Structured and Physically Grounded Visual Models	12
2.5 VLMs and Code Generation for Robotics	14
2.6 Executable Code World Models and VDAWorld	14
2.7 Summary and Research Gap	15
3 Methodology	19
3.1 Problem Formulation	19
3.2 World Modelling System Overview	20
3.3 Visual-Action Demonstrations and Generation Evidence	22
3.4 Explicit Simulator Interface and State Representation	22
3.5 Agentic World Model Generation	23
3.6 Executable Validation, Repair and Simulator Selection	25
3.7 Planning with the Generated Simulator	26
3.7.1 The Planner: Cross-Entropy Method	27

3.7.2	The Controller: Model Predictive Control	27
3.7.3	Summary of Methodology	28
4	Experimental Setup	29
4.1	Benchmark Environments	29
4.2	Demonstration Datasets and Composition	30
4.3	LeWM Comparison and Protocol Matching	32
4.4	Phase 1: Simulator-Generation Protocol	32
4.5	Phase 2: Closed-Loop Evaluation Protocol	33
4.6	Metrics and Statistical Analysis	35
4.6.1	Statistical Procedures	36
4.7	Implementation and Reproducibility	37
5	Results and Analysis	39
5.1	Internal Gate Outcomes Across Benchmarks (RQ1)	39
5.2	Benchmark Case Studies: Generation to Control (RQ1–RQ3)	40
5.3	From Internal Consistency to Closed-Loop Control (RQ2a)	50
5.4	Demonstration Composition, Variance and Auditability (RQ3)	52
5.4.1	Does a particular mix of demonstrations improve world model creation? (RQ3a)	52
5.4.2	Does re-running the same setup give the same result? (RQ3b)	52
5.4.3	Can inspection pinpoint why a simulator failed? (RQ3c)	53
5.5	Summary of Findings	53
6	Discussion	55
6.1	Interpretation and Comparison with World Modelling Literature	55
6.2	Critical Reflection on Methodology	57
6.3	Future Work	58
7	Conclusion	59
	References	61

List of figures

1.1	Explicit world-model workflow	2
1.2	Simulator generation and control	4
2.1	World-model planning with chess	8
2.2	Action-conditioned latent world-model planning	11
2.3	Planner-facing world-model structure	13
3.1	System architecture	21
4.1	Benchmark environments	30
5.1	Two-Room rollout	43
5.2	Reacher rollout against LeWM	44
5.3	Reacher prediction mismatch	45
5.4	Push-T failure without a pusher objective	48
5.5	OGBench-Cube failure from a 2D state	51
5.6	Phase 1 ratio and Phase 2 success	52

List of tables

2.1	Planner-facing world-model comparison	16
3.1	Generated simulator template	24
3.2	Tools available during simulator generation	24
4.1	Benchmark inputs and success criteria	31
4.2	Demonstration trajectory types	31
4.3	Phase 1 validation gate	34
4.4	Phase 2 planning settings	35
4.5	Core recorded metrics	36
4.6	Statistical procedures	36
5.1	Phase 1 validation results	40
5.2	Closed-loop success by simulator	41
5.3	Paired comparison with LeWM	41
5.4	Results by demonstration composition	53

Chapter 1

Introduction

1.1 Problem Statement

For physical AI to deliver real-world value, robots must anticipate the consequences of their actions before taking them. In manipulation, pushing an object depends on scene geometry, contact dynamics, obstacles and stability. Learning that by physical trial and error is costly and does not scale.

Direct visuomotor policies map observations to actions effectively [29], but expose no queryable model of how the environment evolves. Model-based approaches instead learn or construct a transition mechanism for weighing alternative futures before committing to an action [45]. This dissertation calls such a mechanism a world model: an internal representation of an environment that predicts how it changes if a given action is taken.

State-of-the-art visual world models predict future states in an abstract latent space rather than in named physical variables [63, 34]. This plans well but leaves state and dynamics neither interpretable nor editable, so diagnosing whether a failure lies in perception, dynamics, goal representation or planning requires auxiliary probes. Conversely, world models expressed as inspectable code generally assume symbolic or textual state descriptions rather than visual input [46, 14].

This dissertation asks whether a small offline evidence set (visual–action demonstrations) and a task-agnostic prompt can serve as the input for building a world model that (a) supports goal-directed control on standard world-model benchmarks and (b) remains inspectable, so its perception, dynamics and objective can be examined separately (Figure 1.1).

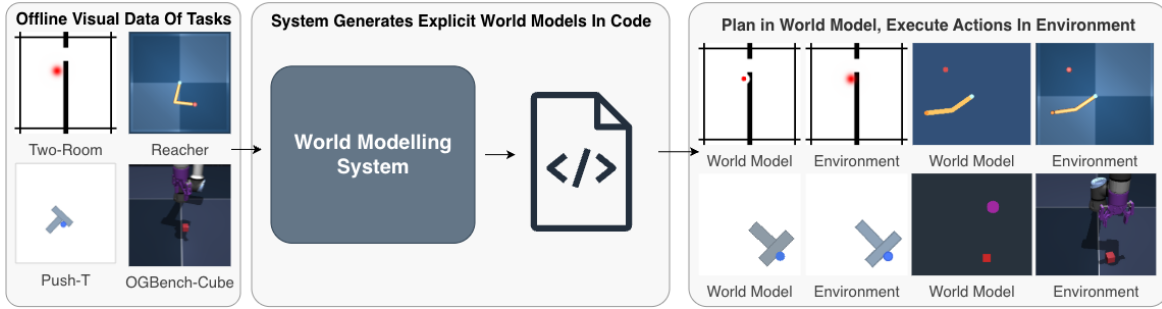


Fig. 1.1 **Explicit world-model workflow.** Across different tasks, the system produces an explicit code world model simulator by viewing images paired with actions that describe what is happening in the environment. A planner chooses actions in the simulator before applying them in the environment.

1.2 Motivation

Vision-Language Models (VLMs), which jointly process images and text and generate code, can inspect visual demonstrations, hypothesise scene structure and write programmes formalising them. Prior work shows such artefacts, whether policies, reward functions or simulations, can be executed, evaluated and revised against automatic feedback [30, 32, 39]. Few demonstrations are needed because the VLM is not asked to learn physics. Its pre-training should teach it that a joint swings and slows, or that a pushed object slides only while contact lasts. The model then uses this physical intuition to identify the relevant objects and fix their parameters.

Code, rather than an embedding, makes the model readable. Named state variables make a mis-parsed position visible rather than inferred from a probe. A wrong gain or missing constraint is a line of code rather than an opaque weight matrix. The objective is a separate function, editable without retraining, expressing task structure a latent distance can not.

Where a latent model can satisfy its objective by collapsing the encoder and ignoring the scene (Section 2.3), a VLM code-generated transition rule can read the goal directly, replaying every demonstration perfectly while predicting nothing. Since the world model is code rather than weights, the safeguard is a runnable test in the form of a validation check the programme must pass before it is deployed. Whether such world models are faithful enough for test-time control, and how they compare with latent ones under an aligned evaluation, is the subject of this dissertation.

1.3 Proposed Approach

Building upon VDAWorld [39], this project extends visually grounded simulator generation to explicit, action-conditioned world models for control. Figure 1.2 shows the two-phase framework.

In Phase 1 (P1), a small offline dataset of images paired with actions serves as the task specification. A VLM infers a structured state representation from this, together with a task-agnostic prompt, perception tools and automatic feedback. It then writes an executable Python simulator world model. The resulting Python simulator must pass stress checks, with failures returned to the VLM for code revision, until it works. It is then frozen.

In Phase 2 (P2), the frozen world model is used for control. Candidate action sequences are optimised with the Cross-Entropy Method (CEM), a derivative-free optimiser that samples sequences, scores rollouts and refits around the best candidates. Model Predictive Control (MPC) [35] executes the plan, re-observes and replans. This process matches the planning family of latent world models such as DINO-WM and LeWM, and the control parameters and budgets of the local LeWM reruns [63, 34]. The difference here is that this project replaces learned hidden transitions with Python functions that are exposed for evaluation.

1.4 Research Questions and Contributions

The three central research questions are:

- RQ1 Model construction and validation:** Can a VLM synthesise an executable simulator passing a fixed set of required internal checks covering replay, objective reduction, perception, structural independence and planner drivability?
- RQ2 Control utility and comparison:** a) How do gate outcomes relate to closed-loop control, and b) how do code-based simulators compare with a latent model?
- RQ3 Robustness and auditability:** a) How are demonstration compositions and b) repeated generations associated with the results, and c) what clear failures can code inspection support?

The contributions are:

- A framework that a code-writing VLM can build against and a planner can then use, requiring everything the simulator tracks to be readable from a single image.

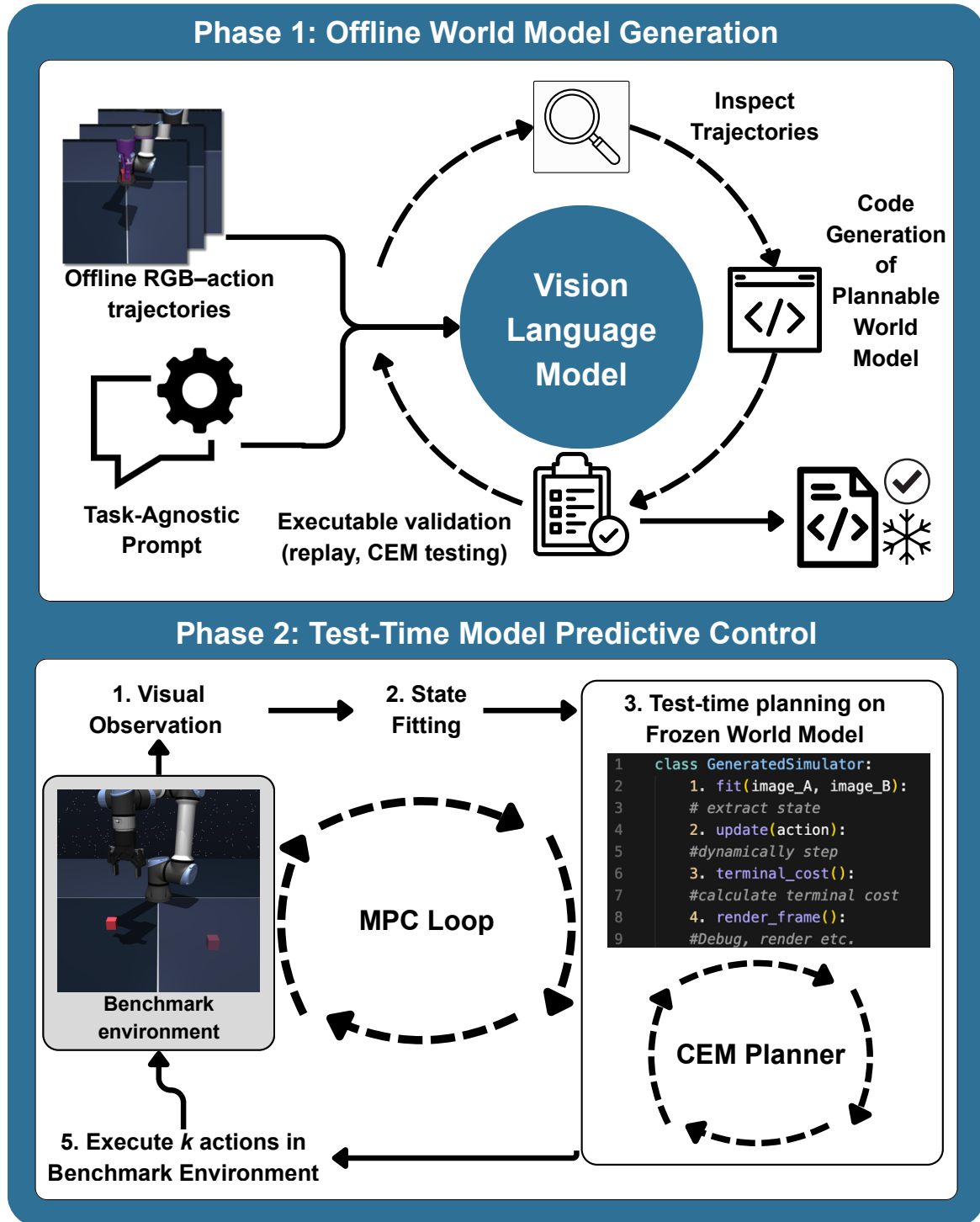


Fig. 1.2 **Simulator generation and control.** In Phase 1, a VLM uses offline RGB-action trajectories and a task-agnostic prompt to generate, validate and freeze an executable world model. In Phase 2, CEM searches for actions in that model, while MPC executes actions in the environment, observes the outcome and replans.

- A set of validation checks that test whether demonstrated actions reduce the simulator’s own cost and whether a planner can drive it.
- An evaluation across four standard latent-world-model robotics benchmark environments in 2D and 3D, including navigation tasks (Two-Room [42] and Reacher [48]) and manipulation tasks (Push-T [10] and OGBench-Cube [40]), each compared against LeWM run locally on the same cases.
- A code-level review that traces each control failure back to the part of the simulator responsible.

In summary, the generated code-based simulators matched or outperformed LeWM on the Two-Room and Reacher tasks, but were decisively beaten by LeWM on the Push-T and OGBench-Cube manipulation tasks. The failures were caused by factors the VLM never made explicit, and no internal check could register the absence. Repeated runs of an identical configuration also varied widely, so the pipeline produces a distribution of world models rather than a consistent output.

1.5 Dissertation Structure

1. **Background and Related Work** (Chapter 2) reviews latent, structured and executable world models to locate the research gap.
2. **Methodology** (Chapter 3) specifies the world model synthesis and repair loop, the internal validation checks and the CEM/MPC controller.
3. **Experimental Setup** (Chapter 4) defines the benchmarks, demonstration compositions, Phase 1 gate and aligned LeWM baselines.
4. **Results** (Chapter 5) evaluates simulator performance across four benchmarks against LeWM. Case studies explore the gap between offline validation and closed-loop control, alongside demonstration composition, generation variance, and code inspection.
5. **Discussion** (Chapter 6) interprets the results with regard to the literature, along with the limitations of the system, what would have been done differently and avenues for future work.
6. **Conclusion** (Chapter 7) closes on what the work established and what must follow.

Chapter 2

Background and Related Work

2.1 World Models as Internal Predictive Models

The “world model” concept originates in cognitive science, where Craik [13] argues that the mind builds internal models of reality to test outcomes before acting. A chess player deciding on a move (Figure 2.1) simulates sequences across a representation of the board and selects the best action: the “world” is an internal model, not the physical board. A world model is defined by this ability to reason about what *would* happen. Sutton’s Dyna architecture [45] formalised this computationally, with experience updating a predictive model and simulated transitions supplementing it during planning. This matters in robotics, where physical experience is slow, expensive or unsafe. Modern world models keep the same triad, a state representation, a transition model predicting its evolution under an action, and an objective for scoring candidate futures.

2.2 Model-Based Planning and Model Error

Ha and Schmidhuber popularised the modern concept of learning control through latent imagination in their paper *World Models* [19]. Their architecture separates representation, dynamics and control. A variational autoencoder (V) compresses each visual observation into a compact latent state. A recurrent mixture-density network (M) maintains information about previous states and predicts a distribution over the next latent state. Finally, a compact controller (C) maps the latent state and recurrent memory to an action. Restricted to a linear mapping from latent state and recurrent memory to actions, the controller relied on the V and M components to provide task-relevant representations and predictions rather than learning perception or dynamics itself. Although these experiments demonstrated that

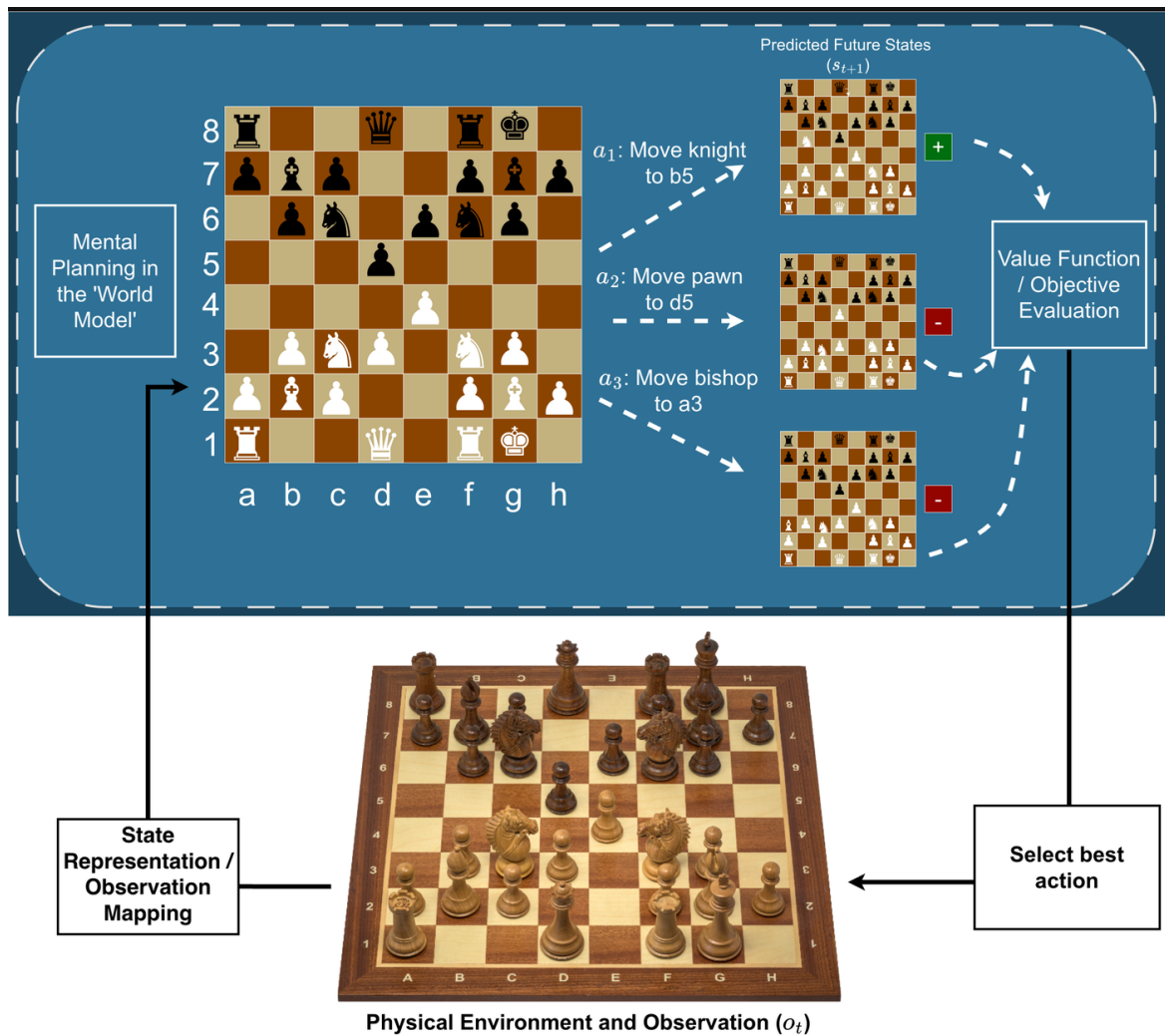


Fig. 2.1 **World-model planning with chess.** The observed board is mapped to an internal state. Candidate moves produce predicted future states, which are scored so the best move can be applied to the physical board.

latent imagination could support control, they were confined to simulated domains such as CarRacing-v0 and VizDoom. Robotics introduces additional challenges, including noisy observations, contact dynamics, object interactions and partial observability. Across the broader literature, learned world models have supported control in two principal ways: through direct test-time planning and through policy learning from imagined trajectories.

Deep Visual Foresight [16] exemplifies the first, using action-conditioned video prediction and MPC to select pushing actions on physical robots. PlaNet [21] moved planning into a compact latent space, optimising rollouts of learned dynamics with CEM in a receding-horizon controller [15]. Dreamer [22] instead trained actor and critic networks on imagined latent trajectories, avoiding action search at deployment, and DayDreamer applied this to physical robots [58].

These approaches remain limited by model error. Autoregressive prediction can accumulate error over long horizons, while a trajectory optimiser may actively seek actions in regions where the learned model is inaccurate. PETS [11] addresses this problem using an ensemble of probabilistic dynamics models, allowing uncertainty to influence trajectory optimisation and reducing reliance on unsupported predictions. Short-horizon model predictive control provides a complementary mitigation by executing only part of a plan before obtaining a new observation and replanning. More recent systems such as TD-MPC2 [23] combine short-horizon latent trajectory optimisation with learned value estimates, although their representations and objectives are trained using task reward supervision.

A low prediction error average does not necessarily indicate successful control. A planner deliberately searches for action sequences that the model predicts will succeed and may therefore exploit small errors in its dynamics or objective. World models should be evaluated through downstream planning success as well as predictive fidelity. The following section examines how recent latent visual world models construct representations and dynamics for planner-facing control.

2.3 Latent Visual World Models and JEPAs

Visual observations mix task-relevant structure with detail such as texture and lighting that control may not need, so many planner-facing models predict compact latent states rather than pixels. Generative video world models take the opposite route and predict frames directly, with Genie [7] learning a latent action space from unlabelled internet video to make generated environments controllable frame by frame and UniSim [59] training an action-conditioned video diffusion model whose rollouts support policy learning that transfers to real robots. Both rely on internet-scale video and per-step sampling that is data and computationally

costly. Despite that cost, their predicted state is still only a 2D image, which exposes nothing a planner can query. The contact, occlusion and depth that decide an action’s outcome remain implicit in pixels rather than available as named quantities.

Systems such as PlaNet [21] and Dreamer [22] learned these states partly through observation reconstruction and reward prediction. While this yields a rich learning signal, it risks allocating model capacity to visually salient details that do not affect action outcomes or planning.

Reconstruction-free approaches instead train predictors directly in representation space. MuDreamer [8] and subsequent joint-embedding methods exemplify this shift. Their objective is to preserve information whose temporal evolution is useful for prediction and control, not to reproduce every pixel.

The joint-embedding predictive architecture (JEPA) formalises this reconstruction-free direction [28]. An encoder maps observations into an embedding space, and a jointly trained predictor, given the current embedding and a conditioning variable describing how the world changes between inputs, predicts the embedding of a related input such as the next frame. The loss compares this with the encoder’s embedding of the actual next input, so prediction and supervision both live in the learned embedding space. Lacking a decoder or pixel-level target, the encoder may discard appearance detail irrelevant to how the scene evolves, saving computational resources. In the action-conditioned form, used for control, the conditioning variable is the agent’s action, and predictions are made in the latent world model.

This latent space objective allows the representation to discard pixel detail unnecessary for modelling the observed dynamics. However, this objective allows a trivial collapsed solution. If the encoder maps every observation to the same embedding, the predictor can minimise its loss without learning meaningful state distinctions or dynamics. Self-supervised JEPAs such as I-JEPA and V-JEPA prevent this collapse with asymmetric target encoders and stop-gradients [1, 4]. The most relevant action-conditioned models take one of two routes (Figure 2.2), either training the encoder end-to-end under an explicit anti-collapse regulariser or freezing a pretrained encoder so targets can not degenerate.

PLDM [42] takes the first route, training its encoder and latent dynamics end-to-end from reward-free offline data under a variance–invariance–covariance regularisation (VICReg) objective whose variance and covariance terms prevent collapse. Its cross-dataset comparison shows latent planning depends on dataset quality. This weakness motivates this dissertation’s composition study.

LeWM [34] is a JEPA in the strict joint-embedding sense, training encoder and predictor together under a next-embedding loss plus the Sketched-Isotropic-Gaussian Regularizer (SI-GReg), which pushes embeddings towards an isotropic Gaussian. Its ~ 15 -million-parameter

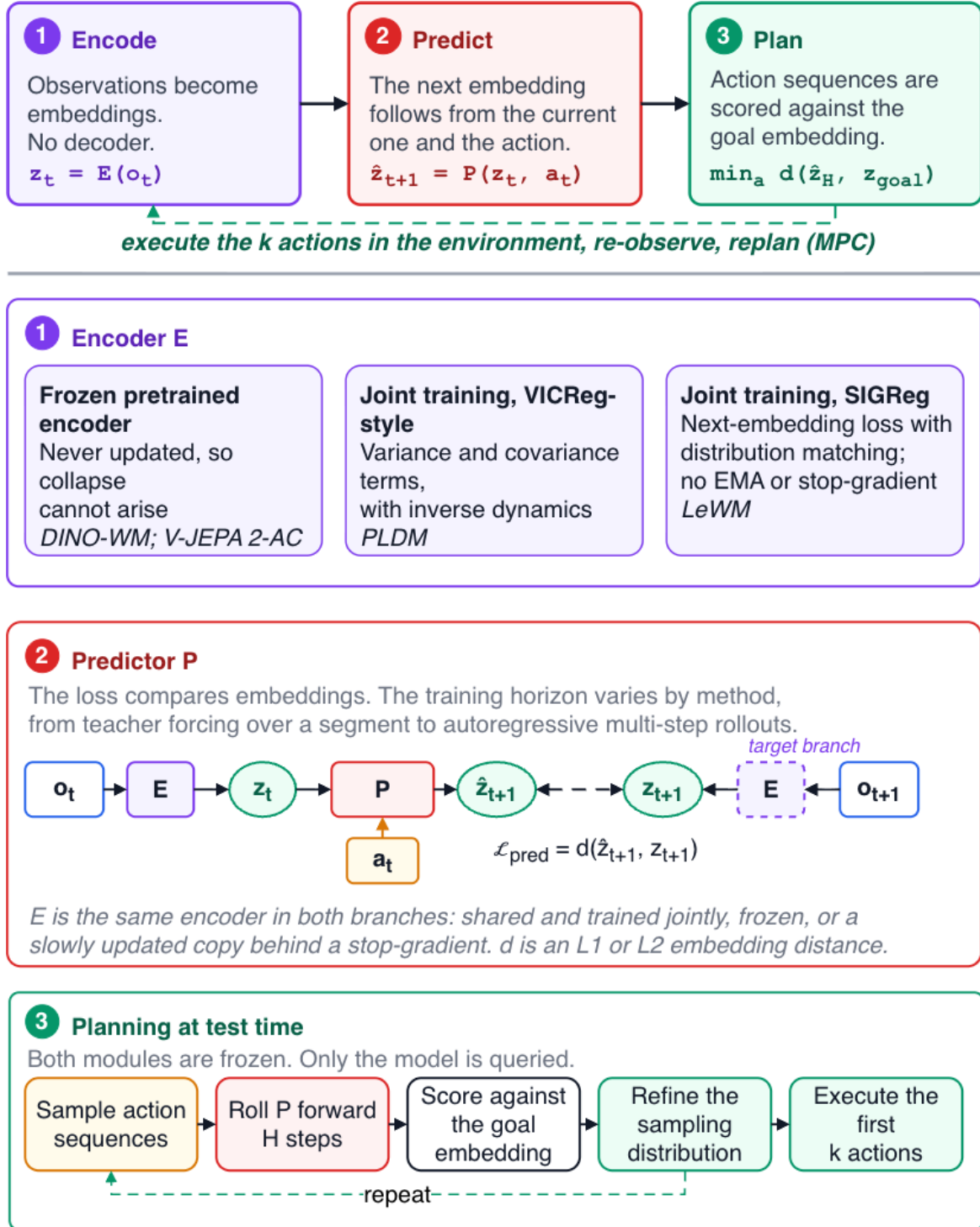


Fig. 2.2 **Action-conditioned latent world-model planning.** Images are encoded as latent states, a predictor rolls those states forward under candidate actions, and distance to an encoded goal scores each rollout. A frozen encoder or anti-collapse training keeps the representation useful, while test-time planning repeatedly samples, scores and executes action sequences.

model trains on one GPU within hours, plans in under one second and is evaluated on Two-Room, Reacher, Push-T and OGBench-Cube. This makes it the closest protocol-relevant latent model, rerun locally for comparison with this project’s results. Sub-JEPA [62] extends it with Gaussian regularisation across random orthogonal subspaces.

DINO-WM [63] takes the second route, freezing pretrained DINOv2 spatial features and training a Vision Transformer dynamics predictor over them. Sharing the embedding-space objective without joint training, it is better described as a latent world model over pretrained features, not a JEPA. Freezing prevents collapse by construction and supplies strong spatial priors, but prevents the representation adapting during dynamics training. At test time CEM searches for actions minimising distance to an encoded visual goal, so new goals need no reward learning, inverse model or fine-tuning. DDP-WM [60] separates sparse primary dynamics from cheaper context-dependent background updates, reporting substantial inference speed-ups and an increase in Push-T MPC success from 90% to 98% relative to its DINO-WM baseline. Temporal Straightening [55] instead regularises latent trajectory curvature, making Euclidean embedding distance a better approximation to geodesic reachability and improving gradient-based planning. Both are evaluated using different benchmarks, which differ from the LeWM ones. Their results demonstrate improvements over their respective DINO-WM baselines, but have not established superiority over LeWM or Sub-JEPA.

At larger scale, V-JEPA 2-AC [2] pretrains on over one million hours of video before post-training an action-conditioned predictor on under 62 hours of unlabelled robot trajectories, planning to image goals on unseen Franka robotics setups [20]. This scale makes it contextual rather than a direct comparison to what is done in this project. Terver et al. [51] find encoder choice, proprioception, multi-step losses, action conditioning, context length and planner all materially affect JEPA planning, so performance depends on the interaction between representation, dynamics, objective and planner rather than any single regulariser.

Recent latent models are therefore increasingly stable, efficient and planner-compatible, but better planning does not make internal states inspectable. Task-relevant variables and transition mechanisms stay encoded in learned representations, and are only examinable through auxiliary probes. This motivates approaches that make the world model representations explicit while retaining the performance of learned prediction.

2.4 Structured and Physically Grounded Visual Models

Structured visual world models occupy a middle ground between dense representations and executable simulators, imposing biases that organise predictions around objects, geometry,

spatial relations or physical primitives. Such structure suits manipulation, where a few entities, poses and contacts determine an action’s outcome (Figure 2.3).

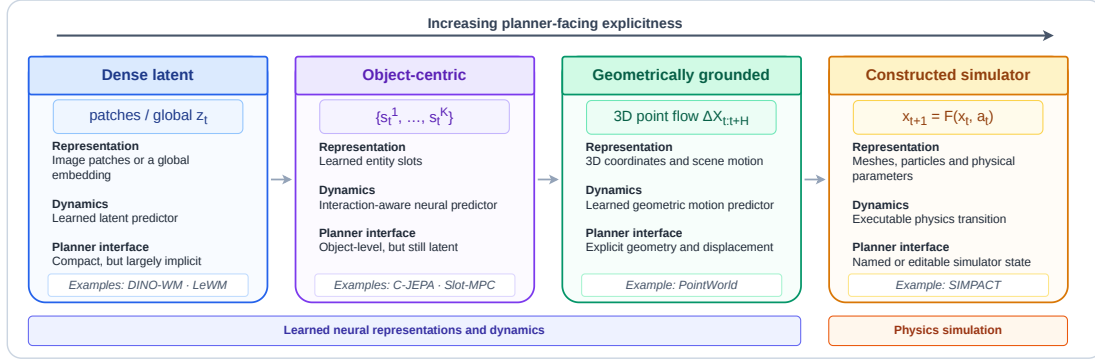


Fig. 2.3 **Planner-facing world-model structure.** This conceptual spectrum runs from dense latent features, through object-based and 3D geometric states, to a constructed physics simulator. Moving right exposes more geometric or physical structure to the planner.

Object-centric models represent scenes as entities rather than undifferentiated tokens. Causal-JEPA [38] masks complete object trajectories so their states must be inferred from surrounding entities, approaching patch-based DINO-WM’s Push-T success using around 1% of its latent input size, while Slot-MPC [43] encodes images into object slots, predicts their action-conditioned evolution and aligns them with goal slots before gradient-based MPC. Both add object-level structure beyond dense embeddings, though their slots remain learned latent variables rather than named, editable physical states.

Geometric models enforce a different grounding. PointWorld [25] represents observations and actions as 3D point flows, predicting scene motion across embodiments and supporting real-time MPC. Learned from roughly two million trajectories, it is a frontier system rather than a data-matched comparator. SIMPACT [31] instead builds a mesh or particle-based physics simulation at test time from a single colour-plus-depth (RGB-D) observation and instruction, with a VLM proposing actions, observing simulated consequences and refining them. These object-centric and geometric models retain learned neural dynamics, and SIMPACT instantiates a simulator for one observed scene, but neither synthesise reusable executable transition code from offline RGB–action trajectories.

2.5 VLMs and Code Generation for Robotics

VLMs support the strategy presented in this dissertation. Their backbones encode objects and spatial relations while their language components produce executable code. Internet-scale image–text pretraining does not itself provide grounded knowledge of how low-level actions change an environment, so robotic systems must connect these capabilities to action and physical feedback.

One approach folds visual and linguistic pretraining into policies predicting actions directly. Vision-language-action models such as RT-2 [6], Octo [49], OpenVLA [27] and π_0 [5] map observations and instructions to motor commands from large multi-robot corpora. They produce capable policies without trajectory optimisation. They do not expose queryable transition models for evaluating counterfactual sequences or attributing failures to perception, dynamics or objective.

A second approach produces executable artefacts, such as ViperGPT [44] and VisProg [18], which use vision tools for visual reasoning. In robotics, Code as Policies [30] generates policy programmes from instructions and APIs, Eureka [32] generates reward functions refined through simulation feedback, and GenSim [54] and RoboGen [56] extend generation to simulation tasks, scenes and skills. Their common advantage is that generated artefacts can be executed, evaluated and revised rather than accepted on trust.

From Pixels to Predicates [3] brings this closer to world modelling. From a handful of short visual demonstrations a VLM proposes predicates describing objects and relations, an optimisation procedure selects a compact predicate set and learns symbolic operators, and a search-based planner composes low-level skills. This is visually grounded and planner-usable, but assumes a skill library rather than predicting continuous low-level dynamics.

2.6 Executable Code World Models and VDAWorld

Executable code world models express predictive state and dynamics as structured programmes and, unlike dense latent predictors, expose explicit interfaces to query states, simulate hypothetical actions and evaluate predicted futures, making representation, transition logic and objective separately inspectable. Code does not guarantee correctness, but it exposes failures explicitly so that they can be fixed.

WorldCoder [46] writes transition and reward programmes from symbolic interaction, using bandit-based refinement to fix code failing to explain observed transitions. GIF-MCTS [14] navigates code edits by Monte Carlo Tree Search on the Code World Models Benchmark (CWMB), and PoE-World [41] combines small probabilistic programmatic experts learned

from brief demonstrations. These show generated code can support planning from limited data, but operate on environment-supplied symbolic or object-level observations rather than discovering state from raw images.

Recent systems begin to connect executable simulation with visual grounding. *Coding Agent Is Good As World Simulator* [53] coordinates planning, code-generation, visual-review and physics-analysis agents to build PyChrono [47] simulations from language prompts and optional images, repaired with visual and physical feedback. SIMPACT and From Pixels to Predicates likewise ground simulation and symbolic operators in images, but target prompted scenes or predefined skills rather than a reusable low-level transition programme learned from offline image–action trajectories.

VDAWorld [39], the precursor to this project, equips a VLM with perception tools and an execution-and-critique loop converting a single image–caption pair into a scene abstraction and executable simulator. It demonstrates manually specified interventions but provides no autonomous planning, since its transition logic is not conditioned on a continuous action space learned from offline data, it exposes no goal-conditioned objective, and it is neither integrated with sampling-based solvers nor evaluated in closed loop.

This dissertation supplies that conjunction, and Table 2.1 places it against the reviewed systems.

2.7 Summary and Research Gap

No reviewed system synthesises explicit executable dynamics in a code world model directly from visual RGB–action trajectories, then evaluates the result in benchmark environments standard for latent world models. Three gaps follow:

1. **Visual-to-executable construction.** Programmatic world models rely on pre-parsed symbolic states, whereas visually grounded models are restricted to passive dynamics or single-scene physics instantiation, so jointly inferring task-relevant state and executable continuous-action transitions from raw visual trajectories remains undemonstrated.
2. **Planner utility and protocols.** Symbolic models plan in discrete games and latent models solve visual benchmarks, but whether generated programmes can drive goal-conditioned CEM/MPC under comparable evaluation is unclear. Answering that requires explicitly aligning task instances, action scaling, control budgets and success criteria.
3. **Reliability and component-level inspection.** Code generation is stochastic and internal replay performance does not imply closed-loop success, so systems should

System	Task-specific grounding data	Explicit programme	Continuous actions	Planner-facing objective	Closed-loop evaluation	Inspectible failure
<i>Programme synthesis from interaction or symbolic observations</i>						
WorldCoder [46]	Online interaction, symbolic	✓	✗	✓	✓	✗
GIF-MCTS (CWMB) [14]	Text description, of-line trajectories	✓	▲	✓	✓	✗
PoE-World [41]	Few object-centric trajectories	✓	✗	▲	✓	✗
<i>Visually grounded or VLM-directed synthesis</i>						
Pixels to Predicates [3]	Visual demonstrations, predefined skills	✓	✗	✓	✓	✗
Coding Agent Sim. [53]	Text prompt, optional image	✓	✗	✗	✗	✗
SIMPACT [31]	Single RGB-D frame, instruction	▲	▲	▲	✓	✗
VDAWorld (v1) [39]	Image-caption pair	✓	✗	✗	✗	▲
<i>Latent world models (paradigm reference)</i>						
LeWM, DINO-WM [34, 63]	~10 ⁴ offline episodes per environment	✗	✓	▲	✓	▲
This project	Eight RGB-action demonstration trajectories	✓	✓	✓	✓	✓

Table 2.1 **Planner-facing world-model comparison.** Reviewed systems are compared by their task-specific evidence and whether they provide an explicit programme, continuous actions, an objective a planner can use, closed-loop test-time evaluation and inspectible failures. ✓ means present, ▲ partial or mixed and ✗ absent.

be evaluated across separate generation runs and data compositions. Explicit code components also make reasons for failure inspectable.

These gaps motivate the following framework.

Chapter 3

Methodology

The method operates in two phases. In Phase 1 (P1), the VLM converts a small offline dataset of visual–action demonstrations into an executable Python simulator **world model**, iteratively testing and repairing its own code against validation checks. In Phase 2 (P2), the programme is frozen and used as the world model inside a Cross-Entropy Method (CEM) **planner** under model predictive **control** (MPC).

3.1 Problem Formulation

The system receives an offline visual–action dataset

$$\mathcal{D} = \{\tau_i\}_{i=1}^M, \quad \tau_i = \left(o_0^{(i)}, a_0^{(i)}, o_1^{(i)}, \dots, a_{T_i-1}^{(i)}, o_{T_i}^{(i)}\right), \quad (3.1)$$

where each $o_t \in \mathbb{R}^{h \times w \times 3}$ is an RGB observation and each $a_t \in \mathcal{A} \subset \mathbb{R}^d$ is the bounded, continuous action executed between consecutive observations. The dataset excludes task captions, reward signals and symbolic states. Generation additionally uses the fixed two-frame sandbox fit pair $\mathcal{F} = (\tilde{o}_A, \tilde{o}_B)$, taken from a separate expert trajectory with no associated action exposed. The pair has one purpose. It supplies the starting or current state, `image_A`, and the goal state, `image_B`, arguments of `fit` while the code is being written and validated, standing in for the current and goal states the planner supplies at test time. Importantly, this means state extraction can be executed and checked before deployment.

Phase 1 (Simulator Synthesis). Given $\mathcal{D}$ and $\mathcal{F}$, a VLM-driven generator $\mathcal{G}$ must produce an executable simulator programme $S = \mathcal{G}(\mathcal{D}, \mathcal{F})$. $\mathcal{G}$ does not receive a task-specific description or any access to the benchmark environment. It must infer benchmark-specific states, transitions and progress semantics from the trajectories, using only the fixed task-

agnostic prompt, the supplied tools and the validation-gate information. This is important because it restricts the system to using only offline methods provided for creating the world model.

Phase 2 (Planning). At deployment the frozen programme exposes three planner-facing capabilities plus a renderer. Given the current observation o_t and goal observation o_g , it extracts explicit states in a shared representation, $(s_t, s_g) = S.\text{fit}(o_t, o_g)$. It predicts the effect of a candidate action through an action-conditioned transition $s_{t+1} = f_S(s_t, a_t)$, and it scores any simulated state against the goal with a scalar terminal cost $C_S(s, s_g)$. The control problem solved at each planning step, over a horizon of H actions, is

$$a_{0:H-1}^* = \arg \min_{a_{0:H-1} \in \mathcal{A}^H} C_S(\hat{s}_H, s_g), \quad \hat{s}_{k+1} = f_S(\hat{s}_k, a_k), \quad \hat{s}_0 = s_t, \quad (3.2)$$

that is, a bounded action sequence whose H -step simulated rollout minimises the terminal cost. This explains how the planner uses the generated world model to choose the action predicted to perform best in the environment.

3.2 World Modelling System Overview

The framework inherits four elements from VDAWorld [39], namely VLM-directed synthesis of simulator code, subprocess-based execution and runtime checks, perception tools during generation, and an iterative execute-critique-revise loop.

This project extends that framework to action-conditioned world modelling. Action-labelled trajectories replace a single captioned image as evidence, and a new action-conditioned simulator contract defines the planner-facing interface of Section 3.4. Automatic checks then replay demonstrations, test for goal-cheating shortcuts and verify that a planner can drive the candidate, feeding those results back as a repair signal and as a final accept/reject gate before deployment through CEM/MPC (Section 3.7).

What carries from P1 into P2 is the generated code world model. Supporting libraries and perception helpers are fixed infrastructure, not VLM-created components. The generator’s state variables, transition rules and cost must all be written into that code, which is then frozen. During P2 it is never edited, re-prompted or fine-tuned, and the VLM is not called during rollouts. This lets offline generation and closed-loop control be measured separately and ensures no P2 observations can influence generation. This makes it so the world model is entirely built from offline observations, not real interactions (Figure 3.1).

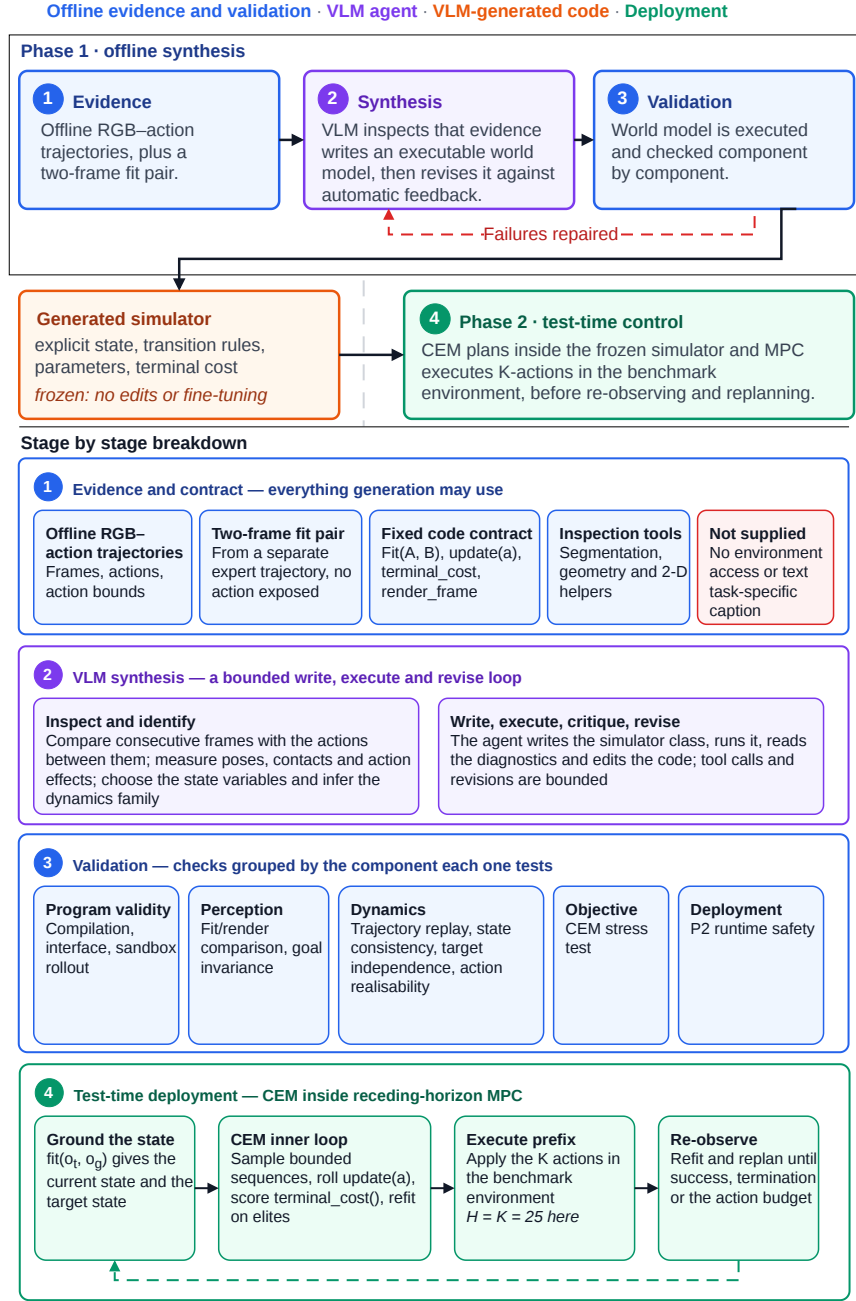


Fig. 3.1 **System architecture.** Phase 1 turns offline RGB–action trajectories into a simulator through VLM code generation, automatic checks and bounded repair. The accepted code is then frozen. In Phase 2, CEM plans in that simulator and MPC executes actions, observes the environment and replans. The lower panels show the evidence, tools, checks and deployment steps in more detail.

3.3 Visual-Action Demonstrations and Generation Evidence

A trajectory with $T+1$ frames contains exactly T actions, and frame t , action a_t and frame $t+1$ together evidence the dynamics linking actions to visual effects.

Demonstrations are in-context evidence rather than imitation targets. Adjacent frames reveal which elements move and which remain static, separating candidate state variables from background, repeated interactions expose constraints such as walls, joint limits and contacts, and final frames of successful demonstrations provide the main goal cue.

During P1 the VLM can inspect per-trajectory summaries, individual frames, the aligned action vectors, automated summaries of per-action visual effects and perception-tool outputs (Table 3.2). Frame size, action dimensionality and observed action ranges are supplied mechanically by the harness as strict interface requirements rather than semantic hints, which prevents benchmark-specific information from entering through the prompt.

Demonstration composition dictates which dynamics are visible. Expert trajectories demonstrate successful completion but explore a narrow state space, whereas noisy and random trajectories expose collisions, limits and other constraints absent from expert paths, and whether the generator uses those trajectories is readable in its code. Partial trajectories withhold the completed goal, preventing the VLM from treating the final frame as a fixed target and pressing it towards general dynamics.

A second expert trajectory per benchmark is stored separately and scored only after generation as a held-out replay diagnostic. Unlike the fit pair it is never exposed through evidence-inspection or fit tools. It never enters generation feedback and never contributes to gate passage. This provides another internal check of correct system behaviour.

3.4 Explicit Simulator Interface and State Representation

Every generated programme must define a class `GeneratedSimulator` with four required methods (Table 3.1), realising a generator-chosen state space $\mathcal{S}$, the transition f_S , the terminal cost C_S and a renderer g_S .

`fit(image_A, image_B)` parses the current and goal observations into the same explicit state space, storing s_t in `self.state` and s_g in `self.target_state`, so observable state must be recoverable from a single image and one parser serves both frames for image-goal planning. During P2 it may optionally receive the previous cycle’s simulated state through a `prev_state` argument, letting the simulator carry forward information hard to see in one frame. A programme opts-in by declaring that argument, which P2 detects from the signature

and passes automatically. This provides it with a useful checkpoint, enabling it to compare where it was last step to where it is heading next.

`update(a)` applies one action to the stored state. It must not read `self.target_state`, because f_S is a function of (s, a) only, so from the same start state the same action sequence must produce the same trajectory whichever goal was fitted.

$$f_S(s, a) \text{ is independent of } s_g. \quad (3.3)$$

Target independence is checked automatically (Section 3.6). Without it, the dynamics could steer towards the goal, behaving more like a controller than a model. This defines the conditions of the environment, not the specific actions to take.

`terminal_cost()` returns a scalar $C_S(s, s_g)$ measuring how far the simulated state is from the goal, which CEM needs to compare candidates. It is generated rather than fixed because each task defines progress differently and a hand-written reward would leak task knowledge. It must agree with `update(a)`, so states with low C_S must be reachable under f_S . If the two disagree, CEM will drive towards low-cost states the dynamics can not actually produce, and the resulting plan optimises the objective without moving the simulated state towards the goal.

`render_frame()` draws the stored state. It may be imperfect, since CEM never optimises it, and it supports human inspection when trying to understand failures.

Two constraints connect these methods. All states that can change during a rollout must live in `self.state`, so each rollout is a function of $(s, a_{0:H-1})$ alone and states held elsewhere can not let one rollout affect the next. This is important because CEM can compare candidate action sequences fairly only when every rollout begins from the same state.

The prompt also asks only for the information needed to explain observed motion, since compact states are easier to extract, simulate and inspect, though omitting velocity, contact or an occluded object can make predictions inaccurate. This framework fixes how the simulator is used, while the VLM chooses what its state contains. This structure makes it easier to inspect the code for errors after it has been written.

3.5 Agentic World Model Generation

Generation follows a tool-using conversation within a fixed turn budget, in which the VLM inspects evidence, implements code, executes it, measures the result and revises. Five tool groups are available (Table 3.2).

Method	Input → Output	Role	Principal failure mode
<code>fit(A, B[, prev_state])</code>	two frames, optional prior state → state, target_state	State and target extraction	Mis-parsed poses, stale state, degenerate target
<code>update(a)</code>	action → mutated state	Forward dynamics	Wrong gains or axes, ignored constraints, goal-dependent shortcuts
<code>terminal_cost()</code> — → scalar		Terminal objective	Minima unreachable under update, flat landscape
<code>render_frame()</code> — → RGB frame		Inspection	Render diverges from state, hiding perception errors

Table 3.1 **Generated simulator template.** The four required methods extract current and goal states, predict the effect of an action, score progress towards the goal and render the predicted state. The last column lists common ways each method can fail.

Category	Tools	Purpose
Evidence inspection	<code>read_trajectory</code> , <code>view_image</code> , <code>view_action</code> , <code>analyze_action_effects</code>	Inspect frames and actions, and see how each action changes the scene.
Perception	<code>segment_image_with_gemini</code> , <code>segment_trajectory_with_gemini</code> , deterministic 2D geometry toolbox	Identify objects and estimate positions, shapes and relationships.
Code construction	<code>write_code_from_scratch</code> , <code>edit_code</code> , <code>read_code</code>	Create, inspect and revise the simulator code.
Execution	<code>check_compilation</code> , <code>run_simulation</code>	Check the code runs and return errors or stack traces.
Validation	<code>view_fit_comparison</code> , <code>validate_against_training</code> , <code>validate_all_trajectories</code> , and per-check tools for state consistency, goal invariance, target independence, action realisability and CEM	Test whether state, dynamics and cost match the demonstrations and support planning.

Table 3.2 **Tools available during simulator generation.** The VLM can inspect demonstrations, estimate scene geometry, write and run code, and test whether the resulting simulator supports planning.

Since segmentation tools are available only during P1, useful information must be built into the simulator’s own image-processing code, keeping deployment self-contained and ensuring P2 evaluates the generated world model rather than continued help from an external VLM. This keeps computational time and costs down.

A single task-agnostic prompt is used across all benchmarks, so per-benchmark prompt engineering can not be disguised as generality. Viewable in the code release, it gives general robotics-task guidance and a fixed tool-usage addendum, and instructs the model to infer state and action semantics before coding, choose the minimum sufficient abstraction, implement deterministic dynamics, avoid filesystem, environment and target-driven access, and produce a terminal objective a derivative-free planner can drive. These are instructions to the VLM, but there are no guarantees it will follow them all.

Before committing each response, the VLM emits intermediate reasoning text. The harness saves this as a per-run `thinking.md`, recording what the generator considered while writing the simulator. Chapter 5 quotes it as evidence, beside the frozen code, because what the model says it intends is not proof of what the programme does. Again, this reflects the move from an opaque latent model to an explicit one.

3.6 Executable Validation, Repair and Simulator Selection

Replaying demonstrated actions is not enough to show a world model is ready for control. A planner searches for promising actions and can exploit places where the model is over-confident [11], so a simulator may replay expert actions well yet fail on shortcuts that work only inside the generated code. Each candidate is therefore run against a fixed *validation battery* of nine checks, testing replay, common cheating behaviours and whether a planner can drive the simulator, without uniquely identifying every failure or showing the model matches environment physics.

The main offline check is the *objective-reduction ratio*, measuring whether demonstrated actions reduce the simulator’s own cost when replayed through its dynamics. For trajectory i the sequence is replayed through `update` from the state fitted on the first frame, with the target fitted from the final frame:

$$r_i = \frac{C_S(\hat{s}_{T_i}^{(i)}, s_g^{(i)})}{C_S(\hat{s}_0^{(i)}, s_g^{(i)})}, \quad \hat{s}_{t+1}^{(i)} = f_S(\hat{s}_t^{(i)}, a_t^{(i)}), \quad (3.4)$$

The ratio is defined *only* when the initial cost is positive. Otherwise the replay check fails. Values $r_i < 1$ show the demonstrated actions reduce the simulator’s own objective, while

$r_i > 1$ flags directional inconsistency such as a sign or axis error. Even $r_i \approx 0$ does not establish accurate dynamics, perception or goal attainment, since numerator and denominator are both defined by the generated state and objective, so their magnitudes form no common fidelity scale and later rank associations are exploratory.

This ratio is the *objective reduction* check. Seven further required checks probe failures it can not detect, namely no replay crashes, fit and render, goal invariance, target independence, action realisability, the CEM stress suite and P2 runtime safety. Between them they catch a target extracted from the current observation, a goal leaking into the transitions, unsupported over-reach and planner-facing defects. These eight are the required gate, and Table 4.3 gives each its exact pass rule. A ninth item, rollout–reparse agreement, is the advisory *state consistency* check. It flags whether a state component diverges without separating transition from parsing error, so it never changes gate passage.

Three mechanisms use these checks:

1. After every code change the harness replays three evenly spaced trajectories and reports their ratios, a diagnostic signal while the code is still being edited. A candidate must pass the final gate defined in the Experimental Setup to be gate-valid.
2. Replay ratios are evaluated on every trajectory in the generation set, while more expensive probes use fixed smaller samples. If a required check fails the generator receives an itemised report and may use bounded repair iterations. An unrepaired programme is retained but marked a P1 failure, excluded from confirmatory claims and used in P2 only for exploratory analysis.
3. Selection is *keep-best* rather than keep-last. The harness ranks checkpoints by required failures then replay ratios, with the strongest few reranked using a reduced CEM probe, instead of always accepting the final edit. A restored checkpoint must still pass the full gate.

Together, these mechanisms should increase the likelihood that the final generated simulator is valid and useful for planning.

3.7 Planning with the Generated Simulator

A world model becomes a controller only when coupled to a planner. Like DINO-WM, PLDM and LeWM [63, 42, 34], this project uses CEM for MPC, with one planner implementation held fixed across all generated programmes and the baseline alignment set out in Section 4.3.

3.7.1 The Planner: Cross-Entropy Method

CEM [15] is used because it is derivative-free. CEM only samples sequences and scores their rollouts, so it needs no gradients. The generated dynamics are Python programmes rather than differentiable neural networks, and may include friction, collisions or contact rules.

The planner maintains a per-step diagonal Gaussian over bounded sequences $a_{0:H-1} \in \mathcal{A}^H$, which scales linearly with Hd and suits repeated MPC, at the cost of ignoring correlations between action dimensions or time steps. The mean starts at the midpoint of the action bounds and the standard deviation at a fixed fraction of the half-range so initial samples cover the admissible space, with clipping guaranteeing deployable actions. Each iteration j draws N_{CEM} candidates, rolls each through the simulator and scores by terminal cost alone. The elite set $\mathcal{E}_j$, the $\max(1, \text{round}(\rho N_{\text{CEM}}))$ lowest-cost sequences for elite fraction ρ , refits both moments:

$$\mu_{j+1} = \frac{1}{|\mathcal{E}_j|} \sum_{a \in \mathcal{E}_j} a, \quad \sigma_{j+1} = \text{std}(\mathcal{E}_j) + \varepsilon, \quad (3.5)$$

with small ε preventing premature collapse. Three details matter for reproducibility. Each candidate rollout starts from a fresh copy of the fitted state and restores it afterwards, so one sample can not affect the next. Candidates are scored only by C_S after H transitions, with no intermediate costs summed, consistent with Equation 3.2. The planner then returns the single lowest-cost sequence seen across all iterations rather than the final elite mean, protecting against late-iteration degeneration.

The main limitation is dimensionality. The search space grows as $|\mathcal{A}|^H$, so longer horizons both dilute a fixed sampling budget and extend the rollout length over which simulator error compounds, and since the planner exploits optimistic errors, longer horizons give it more room to find them. This makes the planning horizon a central design trade-off between control reach, computational cost and exposure to simulator error.

3.7.2 The Controller: Model Predictive Control

One MPC cycle [35] observes the environment, fits the current and goal observations, runs CEM in the generated simulator over an H -step horizon, executes $K \leq H$ actions of the returned plan, then re-observes and repeats until termination or budget exhaustion. Success is recorded if the benchmark criterion holds at any evaluated step.

A smaller K replans more often, correcting drift sooner at the cost of more planning calls, while $K = H$ trusts the model over the full horizon. The LeWM-aligned comparison uses $H = K = 25$, executing each plan in full before replanning, so all correction comes from the single mid-episode replan at step 25. With $K = H$ this remains receding-horizon MPC,

since the plan is discarded and recomputed after each executed horizon, though the receding horizon is degenerate. For programmes accepting `prev_state`, the new image is combined with carried simulator state.

3.7.3 Summary of Methodology

Overall, the methodology converts offline visual-action evidence into validated executable world models for planning. Benchmark-specific configurations, datasets and protocols appear in the Experimental Setup.

Chapter 4

Experimental Setup

This chapter describes the four benchmark environments, the offline demonstrations given to the system, the latent baseline comparison, the tests each simulator had to pass before deployment, the closed-loop evaluation, and how the results were analysed. Phase 1 (P1) is everything the VLM does before the world model is frozen, from inspecting demonstrations to writing and repairing code. Phase 2 (P2) is the frozen programme planning in the environment. A *run* is one complete generation of one simulator, named by its demonstration set and an index, as in expert-only run 1.

4.1 Benchmark Environments

Evaluation used the four tasks from the LeWM’s suite [34], so the comparison happens on the same environments the latent baseline was built around. They rise in difficulty from 2D to 3D, and from moving the agent itself to the agent moving an object: 2D navigation, planar arm control navigation, contact-rich planar pushing and 3D pick-and-place (Table 4.1, with observations in Figure 4.1).

Both LeWM and this project’s system receive the same basic inputs in every task. These comprise the current RGB image $A \ o_t$ and a goal image $B \ o_g$, the test-time fit pair (Section 3.1). A world model’s quality is measured by how it performs in the test-time environment.

Two-Room [42] is 2D navigation to a target in two rooms divided by a wall with a single doorway. The straight line to the target is usually blocked, so the task tests whether a model captures a hard geometric constraint instead of assuming free Euclidean travel. It uses the LeWM Stable-WorldModel (SWM) environment [33] at 5 px per step, and counts success within 16 pixels of the target, the threshold LeWM’s harness applies.

Reacher [48] is a 2D two-joint planar arm from the DeepMind Control Suite, commanded by joint torques rather than positions, so a model must represent momentum as well as geometry. Success requires the fingertip to reach inside the rendered red target sphere, of radius 0.015 m.

Push-T [10] is contact-rich 2D manipulation, a circular pusher steering a T-shaped block to a target pose. It is the most demanding test of generated dynamics, since the block moves only while in contact, so behaviour changes abruptly at the pusher boundary and small contact errors compound along a rollout. The fraction of the target region the block covers is also recorded as a secondary metric of success, treated as success at 0.95, separating a near miss from a complete failure, though this is not the metric we use as success in the evaluation.

OGBench-Cube [40] is 3D manipulation, moving a cube to a target position through a five-dimensional arm-and-gripper action. Depth must be inferred from one perspective view in which the gripper can occlude the cube, so visual state extraction is far harder than in the planar tasks.

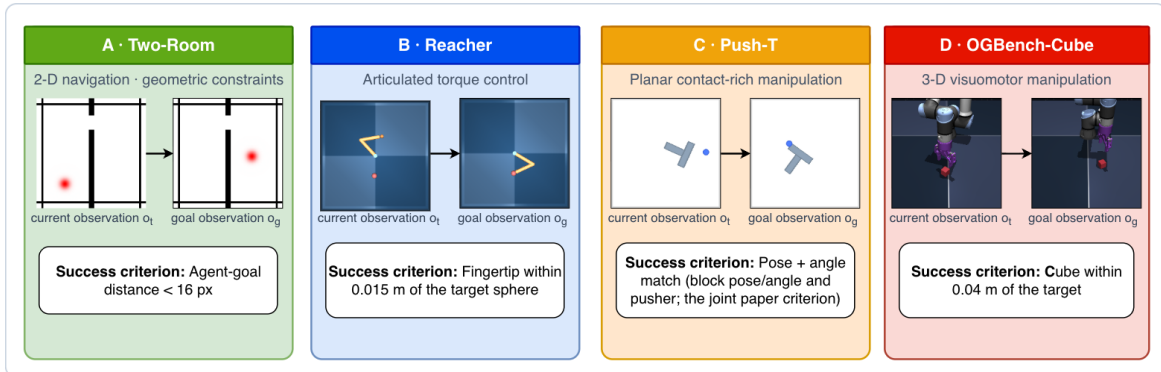


Fig. 4.1 **Benchmark environments.** Example current and goal images for Two-Room, Reacher, Push-T and OGBench-Cube. Each panel also summarises the image input, action space and evaluation measures for that task.

4.2 Demonstration Datasets and Composition

The VLM sees eight trajectories during generation. Each set is called a cell. The cells were built per benchmark environment from four kinds of trajectory, each exposing what the others hide, see Table 4.2 for details.

Six demonstration cells were assembled per benchmark, each holding exactly eight trajectories drawn deterministically, namely `expert_only` (8E), `expert_noisy` (4E+4N),

	Two-Room	Reacher	Push-T	OGBench-Cube
Task	2D navigation	2-joint reach	contact pushing	3D pick-and-place
P1 frame size	224 ²	224 ²	224 ²	200 ²
Action dim.	2	2	2	5
Action semantics	velocity dir.	joint torques	target position	arm + gripper
Median expert trajectory length (frames)	38.5	40	66	63.5
Held-out replay trajectory	expert #9	expert #9	expert #9	expert #9
Success criterion	dist. < 16 px @224	fingertip ≤ 0.015 m	pose + angle match	cube ≤ 0.04 m

Table 4.1 **Benchmark inputs and success criteria.** The four tasks differ in control problem, Phase 1 image size, action space, demonstration length and test-time success measure. The held-out replay trajectory is not shown to the VLM during generation.

Trajectory type	Intent	Two-Room	Reacher	Push-T	OGBench-Cube
Expert (E)	Clear demonstration	Scripted controller	Proportional-derivative (PD) controller	Top-reward LeRobot episodes [9]	Scripted programme
Noisy (N)	Noisy movement towards goal	Perturbed, successful	Noise $\sigma=0.3$ annealed to zero	120-action <i>unsuccessful</i> rollouts	Noise $\sigma=0.3$; successes kept
Random (R)	Random movements in the environment	50 uniform actions	Goal-ignoring joint walk	120 uniform actions	60 uniform actions
Partial (P)	To separate progress from completion	Pre-success prefix	Goal at 60% of displacement	Pre-success actions	$\approx 55\%$ of rollout

Table 4.2 **Demonstration trajectory types.** Expert, noisy, random and partial trajectories expose different behaviours in each benchmark. Each generation set contains eight trajectories drawn from these four types.

expert_partial (4E+4P), expert_random (4E+4R), random_partial (4R+4P) and all_four (2E+2N+2R+2P).

The 8th trajectory, an expert, is reserved and never used as a demonstration. This last expert trajectory supplies a *fit pair*, so state extraction can be run and checked while the code is still being written (Section 3.3). This unused expert trajectory is held back and replayed only after generation, giving a measurement on a trajectory the VLM never saw. This enables the VLM to validate what its produced in code, before the code is frozen for P2.

The small quantity of eight trajectories maintains the study within the few-demonstration regime the method targets. Everything else is held constant across the six sets: the prompt, VLM configuration and tools available. What equal counts do not control is trajectory length, total transitions, success rate or state coverage, all of which move with the mix,

so any difference between sets bundles the kind of trajectory with these properties and is read descriptively rather than causally. The expectation tested, following PLDM’s finding that dataset quality governs latent planning [42], is that non-expert trajectories may reveal dynamics and constraints expert paths never visit.

4.3 LeWM Comparison and Protocol Matching

This system is compared against a latent world model directly. LeWM [34] was selected because it is the closest reference point (Section 2.3). It was rerun locally, using its published code, environments and frozen `quentinll/lewm-*` checkpoints with no task-specific retraining, so both systems could be put on identical cases.

The constants are the benchmark environments, the 50 cases, the goal images, the 50-action budget, the 25-step replan interval and the success criteria. The differences are each system’s planner internals and action parameterisation, because forcing a shared parameterisation would handicap one or both. The comparison is therefore aligned on environment and planning budget, and nothing is claimed about training conditions, where the two differ by roughly three orders of magnitude in data.

The local reruns do not reproduce LeWM’s published results everywhere, which constrains how to interpret the results. Push-T came closest, 94% locally against approximately 96% read from the paper’s figure. Two-Room reached 86% against approximately 87%, but the published number uses 100-step goals and a 150-action budget, so the agreement is coincidental rather than a replication. Reacher reached 54% against approximately 86%. This may be due to the local protocol differing slightly from the published one in goal sampling and test-time case selection, though no single difference was isolated. Cube 44% against approximately 74%, partly explained by a local case filter excluding displacements below 0.08 m and permitting only 50 actions, plausibly making the retained cases harder. Both systems face the same filtered cases, so every paired comparison in Chapter 5 is internally valid, but where the local baseline underperforms its published self is due to slight protocol changes made so that this project can be evaluated with LeWM. Published numbers for LeWM, DINO-WM and PLDM [34, 63, 42] are context only, since all three train on around 10,000 episodes per environment against this work’s eight demonstrations.

4.4 Phase 1: Simulator-Generation Protocol

All generation runs used Gemini 3.6 Flash [17] at temperature 0.0, chosen for cost rather than peak capability. A run requires long-context inspection of image sequences, dozens

of tool calls and several repair rounds, making token price high. Temperature zero makes sampling near-deterministic but can not make a hosted model reproducible. 3.6-flash is a moving label with no dated snapshot, so every run records its identifier, backend, temperature and timestamp instead of claiming repeatability. Segmentation used a separate Gemini 2.5 Flash endpoint [12], unavailable at P2 but still a method component, because its output varies between calls and can change the programme that gets written.

Generation ran under a soft cap of 36 conversation turns, extendable by up to three rounds of eight while checks were still failing, giving a hard maximum of 60. A candidate was “accepted” only if all eight required checks of Table 4.3 passed, and that set of checks is called the *gate* throughout. Every threshold was chosen before the results were produced and held fixed across all benchmark environments and demonstration sets, and the checks are meant to be read jointly: the gate is a validation-check as to whether a programme is good enough to deploy. Full tolerances and tie-breaks are archived with the code release. In the table N_{CEM} is the planner’s sample population and n_{cases} the number of evaluation cases.

Every run archives its source, its `thinking.md` reasoning trace (Section 3.5) and a `metrics.json` of logs, tool calls, timings and check outcomes.

Each benchmark environment received twenty independent runs, 80 in total: five expert-only, the principal set, and three from each of the five mixed sets. The imbalance is deliberate to keep computational costs low, because five runs give a usable picture of run-to-run variation where the main claims are made and three are enough to indicate trends between demonstration cells.

Every gate verdict was saved and every frozen simulator evaluated at P2, including those that failed the gate, which is what allows internal validation and deployed control to be compared rather than assumed to agree.

4.5 Phase 2: Closed-Loop Evaluation Protocol

At P2 the generated simulator is frozen and its segmentation access removed, so anything the segmentation tools revealed during generation must already be compiled into the programme’s own image-processing code.

For each benchmark environment, 50 seeds were produced. These vary the placement of components, such as where the red dot begins in Two-Room, the arm configuration in Reacher, placement of the T-Block in Push-T, and the cube and gripper starting position in OGBench-Cube.

All four benchmark environments also share a 50-action budget, with a re-plan at the 25th action. The planner settings appear in Table 4.4.

Check		Pass rule
No replay crashes		Every generation-set trajectory replays without crashing, with r_i finite.
Objective reduction		Demonstrated actions reduce the simulator’s own objective on every trajectory, with $\max_i r_i \leq 0.50$ (Eq. 3.4).
Action realisability		One-step actions at the bounds, midpoint and extremes move state by at most 1.25 times the largest replay displacement.
Goal invariance		Refitting one goal frame against first, middle and final current frames preserves target keys and shapes within a bounded drift, so the goal is read from the goal image.
Target independence		A fixed start and 20 actions under a different goal leave drift within tolerance, and where bounds straddle zero, zero actions must not appreciably reduce the objective. This blocks dynamics that quietly steer towards the goal.
Fit and render		Three frames fit and render within a normalised RGB error bound, with area ratio, overlap and component count also checked for small masks.
CEM suite	stress	Three probes expose a changed, non-latent state component with sufficient objective sensitivity, and five start–goal pairs then run at seed 42, horizon 20 and $N_{\text{CEM}} = 100$, each reducing the minimum objective below 0.9 times its value at the start state, without realisability violations. This asks whether a planner can drive the programme, not whether it replays.
P2 safety	runtime	A source scan and smoke run complete with perception stubbed, exercising renderer, objective, a short CEM run and two update calls.

Table 4.3 **Phase 1 validation gate.** A generated simulator passes only if it meets all eight checks. The checks cover safe execution, replay improvement, realistic action effects, correct goal handling, image fitting, planner use and Phase 2 runtime safety.

LeWM-aligned ($n_{\text{cases}}=50$, frozen)	
Plan horizon H / executed prefix K	25 / 25 low-level actions
Environment-step budget	50
CEM population / iterations / elites	300 / 30 / 30
Action representation	Generated low-level actions / LeWM 5 blocks $\times$ 5 action repetitions
Goal-state source	Frozen manifest
Evaluation backend	SWM v0.1.1 [33] service

Table 4.4 **Phase 2 planning settings.** The generated system is evaluated on 50 fixed test cases with a 50-action budget and a replan after 25 actions. The setup is aligned with the local LeWM comparison, while each system keeps its own action format.

Frames reach the simulator at 224×224 without compression loss and are used unchanged, except on Cube, whose programmes were written for 200×200 input and whose frames are downscaled once by bilinear interpolation. Push-T needs one conversion, because a generated programme requests an absolute pusher target in image coordinates while the environment expects a bounded relative command: for a request $a_{\text{sim}} \in [0, w]^2$ at frame width $w = 224$ and native pusher position $p_t \in [0, 512]^2$, the executed action is $a_{\text{env}} = \text{clip}(((512/w)a_{\text{sim}} - p_t)/100, -1, 1)$. The clip is the caveat, since part of a request can be silently discarded and no clipping-frequency analysis was performed, so Push-T failures can not be divided between the conversion and the simulator. Goal images are the stored future dataset frame, resized for Cube, which avoids a synthetic-render mismatch while preserving exactly the information LeWM receives, and Reacher’s is rendered live at the IK goal pose.

4.6 Metrics and Statistical Analysis

Table 4.5 lists what was recorded. P1 reporting is deliberately narrow, resting on gate pass rates, worst objective-reduction ratios and which required checks failed, because every P1 quantity is computed inside a programme’s own state space and cost function and so shares no scale with any other programme. Progress measures are used only where they separate a near miss from a complete failure. Headline P2 rates carry exact denominators and Wilson 95% intervals.

Metric	Definition
<i>Phase 1: Generation and validation</i>	
Gate pass rate	Runs passing all eight required checks
Objective-reduction ratio	Worst r_i over generation trajectories (Eq. 3.4)
State consistency	Largest divergence between open-loop state and a fresh fit, advisory
Required-check failures	Which gate checks a run failed
<i>Phase 2: Closed-loop control</i>	
Success	Benchmark criterion for successful control
Goal distance	Closest and final, per episode
Benchmark-specific error	Push-T pose and coverage, Reacher fingertip, Cube position

Table 4.5 **Core recorded metrics.** The objective-reduction ratio measures self-consistency on each programme’s own scale rather than physical fidelity, and state consistency is advisory.

4.6.1 Statistical Procedures

Three statistical procedures are used, each answering one question (Table 4.6).

Procedure	What it does	Why it is needed	What it can not support
Wilson interval [26]	The range of true success rates that could plausibly have produced this count out of 50	Fifty cases is a small sample, so the observed rate is not the true rate	Covers stochasticity in which cases were drawn, not variation between generation runs
Exact McNemar test [37]	Whether the cases that only one system solved are split too unevenly to be luck	Shows if an uneven split is real, not an accident of these 50 cases	Says nothing about which system won or by how much, so read the counts; and the four p -values are not corrected for testing four benchmarks
Spearman’s ρ_S [61]	Ranks simulators by worst objective-reduction ratio against rank by P2 success	Whether offline validation checks anticipate P2 closed-loop outcomes	Post hoc with no p -values, and each programme defines its own objective

Table 4.6 **Statistical procedures.** Wilson intervals show uncertainty in success rates, the exact McNemar test compares paired case outcomes, and Spearman’s ρ_S explores whether offline objective-reduction scores are linked to control success. The final column states the limits of each method.

Why the comparison must be paired. Both systems, this project and LeWM, attempt the same 50 cases, so each yields a *pair* of outcomes, either both succeed, both fail, or exactly one does. The agreements say nothing about which system is better, so only the *discordant* cases separate them. McNemar’s test asks one question about those. If the two systems were equally capable, each disagreement would be a coin flip, so how improbable is a split this uneven? A test that treats the two sets of results as independent samples, such as a two-proportion or Fisher test, throws that pairing away and is the wrong tool. The Wilson interval is used because the standard normal approximation yields impossible bounds (outside 0–100%) for extreme proportions.

Why the counts lead and the p -value follows. The quantity of interest is the difference between the two success counts out of 50, reported in percentage points. The p -value is secondary because it is *unsigned*, since a decisive loss produces as small a value as a decisive win. The counts therefore carry direction and magnitude, while the p -value only certifies that the split is unlikely to be an accident of which 50 cases were drawn.

Case-level and method-level claims. The paired comparison is *case-level*, concerning one frozen programme and saying nothing about the other nineteen runs. Generation is stochastic, so a *method-level* statement would need the spread across runs, and Chapter 5 reports every run’s count for that purpose. It is read descriptively rather than tested, since with 20 runs per benchmark and four benchmarks to correct across, no test at this scale could separate the pipeline’s advantage from chance, so no method-level claim is made. Neither analysis allows cross-benchmark comparison, since success means something completely different in each task.

4.7 Implementation and Reproducibility

All experiments ran on the Isambard-AI cluster [36] using NVIDIA GH200 nodes, with LeWM baselines on one GH200 GPU and the generated-system runtime on Python 3.13. The files available at the link in the front matter contain the task-agnostic prompt and the code comprising the system, archived alongside the RGB-action dataset. The VLM is a hosted model with no dated snapshot, so re-running the pipeline will not reproduce the results stated here exactly.

Chapter 5

Results and Analysis

Twenty simulator generation runs were produced for each benchmark environment under Section 4.4’s design, split between the composition cells. Every run produced one frozen simulator, evaluated at P2 on the same 50 cases as the local LeWM runs.

Thirteen runs passed Phase 1, eleven on Two-Room, two on Reacher; none on Push-T or Cube. The case studies use an illustrative “representative” simulator that captures the typical mechanisms of successes or failures of world models on these benchmarks.

5.1 Internal Gate Outcomes Across Benchmarks (RQ1)

Every P1 generation produced executable code. However, only 13/80 passed the full validation-gate (Table 5.1), all for Two-Room and Reacher. Only 2/80 attempts completed within the initial 36-turn soft cap. 78 required an extended turn budget and 23 exhausted the 60-turn maximum, yet extended budgets did not translate into gate-passing simulators. The keep-best harness shipped a restored checkpoint rather than the final edit in 48/80 attempts, indicating that continued VLM iteration worsened rather than improved the simulators. Two-Room’s nine rejections came only from the CEM stress test. On Reacher, 14/20 met the objective-reduction threshold while the CEM test rejected 15/20 and the advisory state check flagged 15/20. Push-T failed objective reduction in 18/20 attempts, often increasing its own cost under demonstrated actions, and Cube failed in all twenty. These checks measure internal consistency, but do not confirm good P2 performance. The relationship between P1 gate-validation and P2 success is examined in Section 5.3.

Benchmark	Pass/attempt	Pass rate	Median worst ratio [range]	Most consequential required/advisory failures
Two-Room	11/20	55%	0.043[0.015, 0.241]	CEM suite failures: 9/20
Reacher	2/20	10%	0.361[0.091, 1.302]	CEM: 15/20; advisory state consistency: 15/20; ratio: 6/20
Push-T	0/20	0%	1.464[0.287, 2.469]	ratio: 18/20; CEM: 15/20; realisability: 9/20
OGBench-Cube	0/20	0%	1.058[0.948, 1.999]	ratio: 20/20; CEM: 19/20

Table 5.1 **Phase 1 validation results**. Pass rates and worst objective-reduction ratios across 20 runs per benchmark. The ratio must be at most 0.5 to pass. Failure counts can overlap; state consistency is advisory and the other listed checks are required.

5.2 Benchmark Case Studies: Generation to Control (RQ1–RQ3)

Table 5.2 reports all simulators. Table 5.3 shows the paired comparisons with the best-performing runs per benchmark, showing whether the system can produce an executable simulator that works well and is competitive with LeWM. The optional previous-state memory mentioned in Section 3.4 was widely adopted: 75 of 80 programmes declared it in fit (Two-Room 18/20, Reacher 20/20, Push-T 18/20, Cube 19/20), so P2 passed them the previous cycle’s simulated state at the mid-episode replan while five refit from images alone. The replan used a fresh image for all programmes, but for 75/80 combined it with carried model state, so mid-episode corrections depended partly on the pipeline’s own prior predictions rather than purely on fresh observations.

Table 5.3 confirms a split along task type: on Two-Room and Reacher the best simulators solved 27 cases LeWM missed while LeWM solved only one the best did not, but on Push-T and Cube the pattern reversed, with LeWM adding 49 unique successes against just two.

Two-Room: Compact Geometry and Strong Control

The best Two-Room simulator solved 50/50 cases (Wilson [92.9, 100.0]%) against LeWM’s 43/50 ([73.8, 93.1]%). Expert-only input was most reliable, with all five attempts producing gate-valid simulators scoring 47–50/50. Mixed dataset cells were less consistent. Expert-

Composition	Run	Two-Room	Reacher	Push-T	Cube
expert-only	1	50✓	46✓	4	12
	2	49✓	45	4	6
	3	50✓	3	3	10
	4	47✓	7	1	5
	5	50✓	45	1	8
expert-noisy	1	49	32	6	9
	2	49✓	44✓	1	6
	3	49	33	10	8
expert-partial	1	42	44	6	6
	2	28✓	32	2	12
	3	47	36	2	9
expert-random	1	50	43	0	5
	2	48	14	7	5
	3	48✓	34	1	11
random-partial	1	33	1	0	8
	2	50	43	0	10
	3	47✓	4	5	9
all-four	1	42✓	30	3	6
	2	49	38	5	7
	3	26✓	30	1	4
LeWM (local, same cases)		43	27	47	22

Table 5.2 **Closed-loop success by simulator.** Each entry is the number of successful cases out of 50 for one generated simulator. ✓ marks simulators that passed the Phase 1 gate. The final row gives the local LeWM result on the same cases.

Benchmark	This project	LeWM	Project-only	LeWM-only	p
Two-Room	50/50	43/50	7	0	0.016
Reacher	46/50	27/50	20	1	< 0.001
Push-T	10/50	47/50	0	37	< 0.001
Cube	12/50	22/50	2	12	0.013

Table 5.3 **Paired comparison with LeWM.** Best performing at P2 versus local LeWM. “Project-only” and “LeWM-only” count cases solved by only one system; these counts are used in the exact two-sided McNemar test. The p -values are not corrected for testing four benchmarks.

partial and all-four each had a median of 42/50 and between them produced the two weakest simulators. With three to five runs per cell this shows a difference in reliability rather than proving composition causes the gap.

The *expert_only/run_1* shows a mechanism consistent with its success. Its state contains only the red-dot position, it detects walls from each episode’s first image, checks each action for collisions, and uses a cost routing through the doorway.

```
def terminal_cost(self) -> float:
    x, y = self.state["x"], self.state["y"]
    tx, ty = self.target_state["x"], self.target_state["y"]
    # Geodesic path cost through central wall gap
    if (x - self.wall_x) * (tx - self.wall_x) < 0:
        d1 = np.sqrt((x - self.wall_x)**2 + (y - self.gap_y)**2)
        d2 = np.sqrt((tx - self.wall_x)**2 + (ty - self.gap_y)**2)
        return float(d1 + d2)
    return float(np.sqrt((x - tx)**2 + (y - ty)**2))
```

Listing 5.1 Doorway-mediated objective, Two-Room *expert_only/run_1*.

Where a Euclidean distance would pull plans into the wall, this cost guides cross-room plans through the doorway, explaining the seven cases solved only by this work that LeWM could not (Figure 5.1).

The lowest-performing simulators expose the same types of failure. Expert-partial run 2 (28/50) and all-four run 3 (26/50) update positions but omit collision checks entirely, whereas random-partial run 1 (33/50) defaults to fixed doorway coordinates on parsing failure. The archived *thinking.md* traces show both collision-omitting programmes identified the requirement during generation, run 2 noting the agent was “*running into the vertical wall*” and run 3 that its update lines “*allow the red dot to move freely through walls*”. Both final simulators still reduced movement to an unconstrained update, $\text{position} \leftarrow \text{position} + \text{action} \times \text{gain}$. This shows a mismatch between what the VLM noticed and what code it retained.

Conclusion. Two-Room demonstrates that it is possible to synthesise an executable world model that surpasses the performance of a latent one, with a fraction of the data. Eight of the eleven gate-passing simulators beat LeWM’s 43/50 (RQ1–RQ2), but three did not, including the two collision-omitting programmes above, so passing the gate did not guarantee good P2 performance. Expert-only input was the most reliable composition (RQ3a/b), and inspection of failing simulators exposed collision defects that Phase 1 missed (RQ3c).

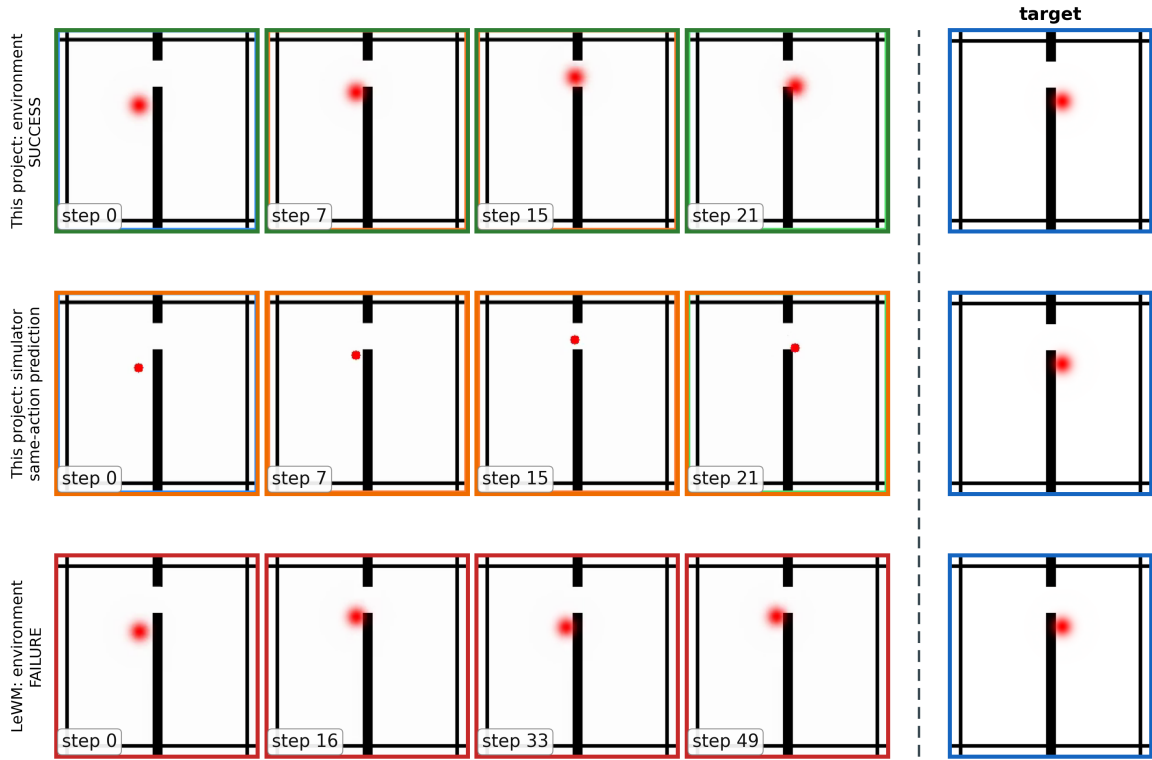


Fig. 5.1 **Two-Room rollout.** In case 65, one of the seven cases LeWM could not solve, this work crosses the doorway and reaches the goal by step 21, while LeWM remains in the starting room. Rows show this work in the environment, its prediction under the same actions and LeWM in the environment.

Reacher: Goal Alignment and Strong Control

Reacher performed well when it worked. The best simulator scored 46/50 (Wilson [81.2, 96.8]%) against 27/50 for LeWM ([40.4, 67.0]%). Figure 5.2 illustrates a case this work solved and LeWM did not.

Goal extraction was generally accurate. Resolved targets were compared with the sphere centroid in each run’s archived goal image: of nineteen pixel-goal programmes, seventeen resolved within 8 pixels and fourteen within 3, while the remaining programme used joint angles and scored 43/50. Programmes either detect the sphere or fit the goal-frame arm endpoint (Listing 5.2), both encodings targeting the same configuration here.

```
# expert_only/run_1: find the red sphere
def _extract_target_xy(self, img):
    red_mask = (img[:, :, 0] >= 180) & (img[:, :, 1] <= 110) & (img[:, :, 2] <=
    ↪ 110)
```

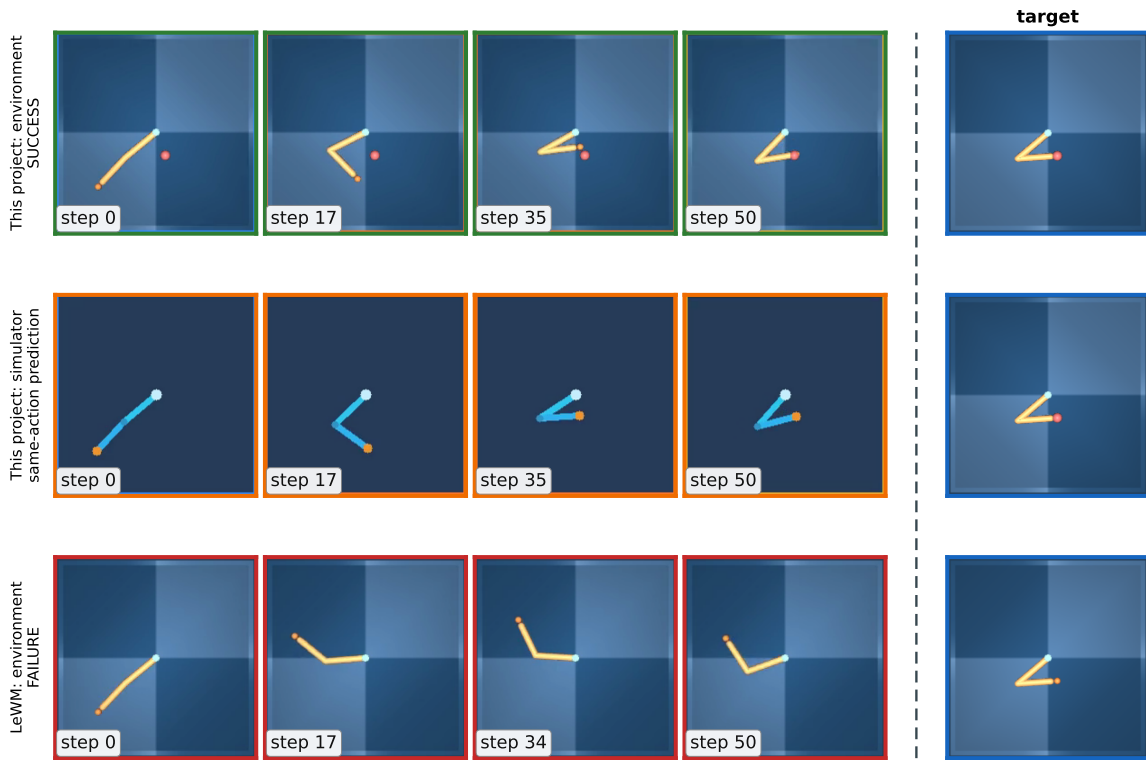


Fig. 5.2 **Reacher rollout against LeWM.** On the same start and goal, this work finishes 0.006 m from the target, within the 0.015 m success radius, while LeWM finishes 0.224 m away. The middle row shows this work's prediction under the same actions.

```

comp = self.select_connected_component(red_mask, min_area=5)
if comp.get("valid", False):
    return np.array(comp["centroid"], dtype=np.float64)
return np.array([62.26, 62.41], dtype=np.float64) # fallback if none found
self.target_state = {"ee_xy": self._extract_target_xy(image_B)}

# expert_partial/run_2: fit the arm shown in image_B
arm_mask_B = self._extract_arm_mask(image_B)
fit_res_B = self.fit_articulated_chain(arm_mask_B, base_xy=self.base_xy,
                                       link_lengths=self.link_lengths,
                                       relative_angles=True)
_, _, ee_B = self.forward_kinematics(*fit_res_B["angles"])
self.target_state = {"ee_x": float(ee_B[0]), "ee_y": float(ee_B[1])}

```

Listing 5.2 The two Reacher goal encodings, both resolving the same point.

Five simulators scored below 15/50. Two mislocated the goal by 47 and 25 pixels, while the other three located it within 5, shifting attention to transition, planning or interface error. Figure 5.3 shows one such case.

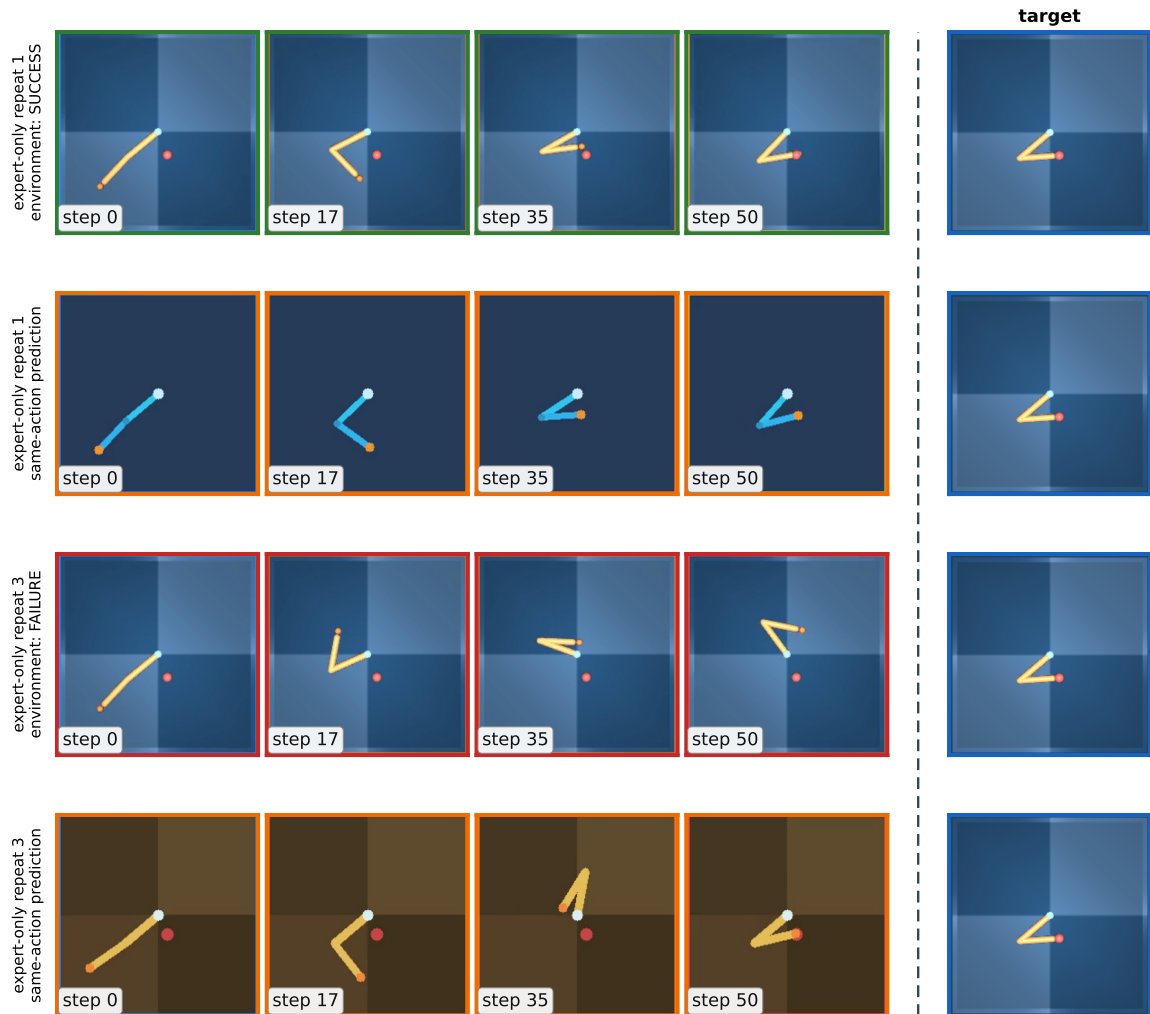


Fig. 5.3 **Reacher prediction mismatch.** In case 46, expert-only run 1 reaches the target at 0.006 m. Run 3 predicts reaching it, but its environment rollout finishes 0.094 m away. Each environment row is followed by that simulator’s prediction under the same actions.

The generated code exposes a reason why. Run 3 applies substantially less shoulder damping relative to its torque gain than the successful example.

```
# expert_only/run_1 (46/50)
gain1, damping1 = 28.0, 6.0
alpha1 = -gain1 * u1 - damping1 * omega1
omega1 += alpha1 * dt_sub
```

```
# expert_only/run_3 (3/50)
k_torque1, damping1 = 16.385, 0.94675
acc1 = k_torque1 * a0 - damping1 * dt1
dt1 += acc1 * sub_dt
```

Listing 5.3 Shoulder dynamics in *expert_only/run_1* and *expert_only/run_3*

Despite different angle conventions, run 3’s damping-to-gain magnitude is about a quarter of the successful example’s, consistent with its simulated arm swinging faster than the real one. The state-consistency check recorded shoulder-angle drift of 135% within six replay steps, and the programme also failed objective reduction and CEM. Its *thinking.md* trace shows the VLM warned itself that low damping could make velocity “*build up far more quickly than expected*”, yet the frozen programme kept weak damping after recalibration. As in Two-Room, the trace identified the defect but the final programme retained it.

Demonstration composition was associated with how programmes represented the goal. Those built from groups containing partial or random trajectories sometimes matched the goal-frame arm endpoint instead of detecting the sphere. Expert-only generations varied from 3–46/50, while all-four scored 30–38/50. With only three to five runs per group, these describe a pattern rather than a causal effect.

Conclusion. Reacher’s best simulators performed strongly, though only 2/20 passed Phase 1 (RQ1). The best scored 46/50 against 27/50 for LeWM (RQ2), and even outperformed the published LeWM result of 86% success. Composition was associated with goal representation and reliability (RQ3a/b). Importantly, when it failed, the code exposed transition-model failures (RQ3c), showing the benefit of this approach compared to an opaque latent model.

Push-T: an incomplete objective and a drifting contact model

No Push-T simulator passed the gate. The best scored 10/50 (Wilson [11.2, 33.0]%) against LeWM’s 47/50 ([83.8, 97.9]%), LeWM solved every case solved by either, plus 37 additional cases. The pattern is consistent with incomplete objectives and drifting contact dynamics.

Nineteen of twenty programmes score only T-block terms. Sixteen omit the pusher from the target state entirely, and three of the remaining four track it but never use it in the cost. The one exception is the best-scoring run, *expert_noisy/run_3*, which does penalise pusher target error. The listing below is therefore taken from a run which tracks the pusher in its state but leaves it out of the objective:

```

self.state = {"pusher_pos": pusher_pos.copy(),          # pusher tracked here
              "obj_pos": obj_pos.copy(), "obj_theta": float(obj_theta)}
self.target_state = {"obj_pos": target_obj_pos.copy(),  # but not here
                    "obj_theta": float(target_obj_theta)}

def terminal_cost(self) -> float:
    d_pos = np.linalg.norm(self.state["obj_pos"] - self.target_state["obj_pos"])
    d_theta = abs(math.atan2(
        math.sin(self.state["obj_theta"] - self.target_state["obj_theta"]),
        math.cos(self.state["obj_theta"] - self.target_state["obj_theta"])))
    return float(d_pos + 20.0 * d_theta)    # no pusher term

```

Listing 5.4 The state carries `pusher_pos` but the target does not.

Once the block is placed, these objective gives no guidance for the pusher. Across the failures, classifying a step as stationary below 0.5 board units of block-centre displacement, the block last moved a median 36 steps before the end while the pusher subsequently travelled a median 1,073 units (Figure 5.4).

In a post-hoc grouping, the four programmes naming the pusher in their target, including the exception `expert_noisy/run_3`, average 5.75/50 against 2.44/50 for the sixteen that do not. The groups are small and differ in more than this choice, so the difference is descriptive. Even `expert_noisy/run_3`, which penalised pusher target error only managed 10/50, meaning the fix was not sufficient.

Objective alignment is not the only limitation. The Push-T target coverage measurement never reached the 0.95 threshold in any of the P2 evaluations, with the best case at 0.89 coverage. The block often approached its target without close overlap. Phase 1 anticipated the instability as objective reduction rejected 18/20 attempts (median worst ratio 1.464), and the advisory state check found block-angle drift in 20/20 programmes and block-position drift in 19/20. The code locates a possible reason, contact being delegated to a generic planar model whose constants are guessed, meaning there is a problem with how the VLM infers the dynamics from the trajectories. The code shows that not enough information could be inferred from the offline dataset for the world model to function properly:

```

res = self.planar_push_step(
    body_polygons=[self.T_TEMPLATE.tolist()],
    ..., kp=15.0, kv=10.0, dt=0.1, substeps=10,
    friction=0.4, damping=0.85, bounds=[0, 0, 224, 224])

```

Listing 5.5 Hand-parameterised contact step, Push-T `expert_noisy/run_3` (abridged).



Fig. 5.4 **Push-T failure without a pusher objective.** In case 87, this work places the block near its target but drives the pusher away, so the full task fails; LeWM reaches both targets. The middle row shows the simulator’s prediction. This behaviour matches an objective that scores the block but not the pusher.

The design did not allow constants to be calibrated against the real-environment. The observed drift is consistent with contact-model mismatch accumulating over each 25-step open-loop segment.

Conclusion. Push-T remains a problem for the system. The best scored 10/50 against LeWM’s 47/50 (RQ1–RQ2). Two deficiencies explain the gap: 19/20 simulators omit the pusher from their objective, and contact-model drift causes the block to reach its target pose but later lose it. Short, inspectable code exposes the failures, enabling identification as to where further improvements need to be made (RQ3c). These results suggest a redesign of the system is required for success on this benchmark.

OGBench-Cube: 2D Representation Did Not Support Picking

No Cube attempt passed Phase 1. The twenty simulators scored 4–12/50 and the best one solved 12/50 (Wilson [14.3, 37.4]%) against LeWM’s 22/50. Both systems face the same filtered cases, so the paired result is valid within the local contract, but LeWM’s 44% is below

the approximate 74% read from its published figure, so the observed loss may understate the gap under that protocol. Two cases were solved by the best and twelve by LeWM, the difference concentrated where the cube had to be picked up.

The problem most likely stems from the VLM representing the scene in 2D rather than 3D. The task was mainly vertical. The median required height change was 0.179 m against 0.035 m horizontally. Cases were classified from the initial cube height, with 26 table starts at 0.019–0.024 m and 24 held starts at 0.053–0.324 m, the 0.05 m split falling in the gap between them. All generated-programme success came from held starts: 156 of 480 held-start evaluations succeeded, none of the 520 table-start evaluations did, and LeWM solved 16/24 held and 6/26 table starts. Figure 5.5 shows `expert_only/run_1` on one case from each side of that split.

In table-start cases the cube moved a median 39 mm across the table but rose only 6 mm, showing the programmes pushed without picking it up. Even most held-start successes needed little horizontal control, with the cube already inside the 0.04 m tolerance in 147 of 156 and only needing lowering. The shared representational limitation is that the programmes treat the scene as a flat image, evident in the prediction rows of Figure 5.5. Nineteen represent the cube in image-plane coordinates without height, and `expert_random/run_1` uses fixed 2D coordinates rather than parsing them. The twentieth declares `cube_z` but keeps it at zero and excludes it from the objective, so none estimates height directly. Fourteen represent grasp or hold status; one uses only a distance threshold and thirteen fit an explicit flag, twelve of which barely change between cases, so these signals say little about whether the cube is held.

```
# expert_only/run_1: image-plane motion, a[3] and a[4] unused
dx = 3.5*a[0] + 6.5*a[1] - 1.5*a[2]
dy = -1.5*a[0] - 1.0*a[1] - 5.5*a[2]
if dist < push_rad and np.linalg.norm(grip_move) > 1e-3:
    cube = cube + grip_move * 0.8          # proximity pushing

# all_four/run_2: grasp is based on two image centroids
finger_xy = gripper_xy + np.array([0.0, 25.0])
dist = float(np.linalg.norm(cube_xy - finger_xy))
grasped = dist < 18.0
```

Listing 5.6 Two Cube programmes: a planar transition ignoring yaw and gripper channels, and a grasp inferred only from pixel distance.

Applied to the archived cases, the centroid-distance test identifies 20 of the 24 genuinely held starts but also labels 21 of the 26 table starts as held, so it can not distinguish the two

situations that matter, as Figure 5.5 illustrates. At the initial fit, eighteen programmes exposed a variable parsed cube-to-goal pixel distance; the remaining two used fixed or degenerate values, which is why this differs from the nineteen image-plane programmes above.

The prediction is therefore hard to diagnose visually: depth movement can appear as little or no change in a flat renderer, which shows what the programme stores rather than what physically occurs. The missing-height interpretation consequently rests on the code, measured 3D reference states and Phase 1 results rather than the prediction images.

Conclusion. No Cube simulator passed Phase 1, and the best solved 12/50 against LeWM’s 22/50 (RQ1–RQ2). All generated-model successes occurred when the cube already began in the gripper: the 2D states supported lowering but not picking, which needs height and a reliable grasp state. No composition enabled picking (RQ3a). The code successfully exposes the omitted representation, meaning the system needs better tools and perception to create 3D world models (RQ3c).

5.3 From Internal Consistency to Closed-Loop Control (RQ2a)

The P1 gate aimed to predict P2 control but was usually uninformative and misleading. Gate passage was not sufficient as the CEM suite admitted two collision-omitting Two-Room models scoring only 26–28/50 while rejecting two others that scored 50/50. Reacher was the clearest positive case, its gate-valid simulators ranking first and near-top on control.

Within-benchmark Spearman correlations between the objective-reduction ratio, how faithfully a simulator replays its own objective under demonstrated actions, and control success were negligible for Two-Room, Push-T and Cube ($|\rho_S| \leq 0.14$) but strong for Reacher ($\rho_S = -0.74$, Figure 5.6). These post-hoc rank associations are not a general quality score, since each programme defines its own objective.

The case studies locate the reason for the unreliability of the internal gate checks. If a world model never represents a quantity at all, such as height in the 3D Cube task, no amount of replaying its own objective can reveal the problem. The checks only asked whether each simulator was consistent with its own variables, not whether those variables matched what the benchmark environment actually needed. Offline analysis can not find a problem the data never contains or does not provide enough information about, so a fully offline pipeline is not sufficient for generating faithful world models, at least in the system’s current state.



Fig. 5.5 OGBench-Cube: the pick is what is missing. Both rollouts come from the same programme, expert-only run 1. In case 44 the cube starts in the gripper at 0.212 m and only has to be lowered, which the programme does, finishing 0.016 m from the target and inside the 0.04 m threshold. In case 86 the cube starts on the table at 0.020 m and has to be lifted to 0.311 m; the programme slides it along the table instead and finishes 0.308 m away, while LeWM picks the same cube up and succeeds. The two prediction rows show what the programme actually represents: flat image-plane markers and a target box, with no height channel.

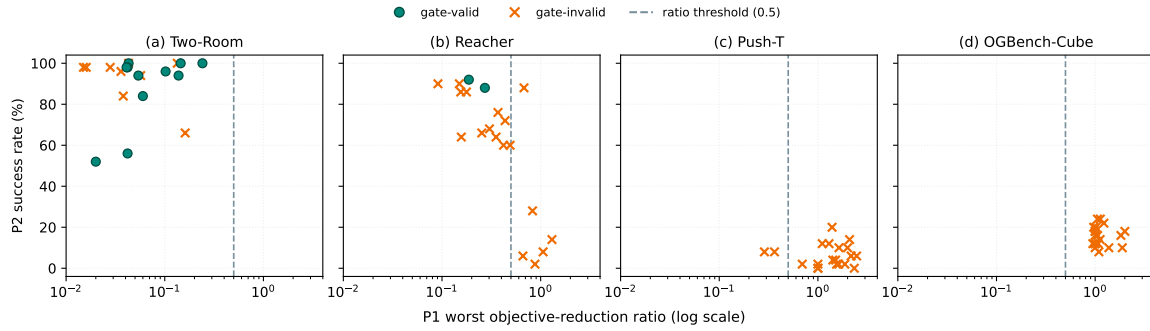


Fig. 5.6 **Phase 1 ratio and Phase 2 success.** Each point is one generated simulator. The x-axis shows its worst objective-reduction ratio on a log scale and the y-axis its closed-loop success rate. The dashed line is the Phase 1 ratio threshold of 0.5; circles passed the full gate and crosses did not.

5.4 Demonstration Composition, Variance and Auditability (RQ3)

5.4.1 Does a particular mix of demonstrations improve world model creation? (RQ3a)

The demonstration mix was associated with simulator reliability for Two-Room, though the answer is no for the other benchmarks. For Two-Room, runs using the expert-only trajectories passed the gate on all five runs and scored 47–50/50 in P2, while every mixed group produced at least one weak run. However, for Reacher, expert-only gave a wide spread of results, with simulators produced having no consistently good world model representation, and therefore highly-varying performance (3–46/50). No mix improved Push-T or Cube, and none was consistent across the board.

These patterns can not be turned into statistical conclusions. With three runs per mixed group, the strongest verdict any test can return is $p=0.10$, and the one standout signal, expert-only passing the gate five times out of five on Two-Room, still rests on five runs. The groups also differ in trajectory length, success rate and coverage, not just label (Section 4.2), so even far more runs would show only that some demonstration sets work better, not which property was responsible.

5.4.2 Does re-running the same setup give the same result? (RQ3b)

No. Repeating the same configuration produced simulators ranging from 26–50/50 on Two-Room and 3–46/50 on Reacher. Temperature 0.0 did not remove this variation, because

the segmentation method, tool-call ordering and other hosted-service non-determinism all change the evidence the VLM sees and the generation path it takes. A single generation is therefore not a reliable sample of the pipeline’s output.

Composition	Runs	Two-Room (%)		Reacher (%)	
		P1	Success (sd)	P1	Success
expert-only	5	100	98.4(2.6)	20	58.4(44.3)
expert-noisy	3	33	98.0(0.0)	33	72.7(13.3)
expert-partial	3	33	78.0(19.7)	0	74.7(12.2)
expert-random	3	33	97.3(2.3)	0	60.7(29.7)
random-partial	3	33	86.7(18.1)	0	32.0(46.9)
all-four	3	67	78.0(23.6)	0	65.3(9.2)
All runs	20	55	90.3(14.8)	10	60.4(31.1)

Table 5.4 **Results by demonstration composition.** Phase 1 columns show the percentage of runs that passed the gate. Success columns show the mean Phase 2 success rate, with sample standard deviation in parentheses. Large standard deviations mean that repeated generations from the same composition produced very different results. Only Two-Room and Reacher are shown because no Push-T or OGBench-Cube run passed Phase 1.

5.4.3 Can inspection pinpoint why a simulator failed? (RQ3c)

Yes, and on every benchmark. Each benchmark’s failure traced to a specific part of the code that was either missing components or guessing information. For example, missing collision checks on Two-Room, too little damping relative to torque gain on Reacher, a cost function with no pusher term on Push-T, and flat image-plane states with no height on Cube (Sections 5.2–5.2). Each of these establish the cause of failure..

5.5 Summary of Findings

The method works on simple navigation tasks. However, complex dynamics are the bottleneck. Where the deciding physics are visible in a flat image, a dot in a walled room or an arm reaching a target, the pipeline produced workable simulators that beat LeWM. Where success depends on contact forces or 3D, the simulators failed. The VLM often flagged defects in its own reasoning traces yet shipped code that retained the errors. What is key is that exposed code showed where each failure originated. The next chapter interprets these limitations against the wider literature.

Chapter 6

Discussion

The results demonstrate that while the proposed approach shows promise, the current system remains constrained by significant limitations. The main contribution is that these limitations can be traced to specific lines of code. The system worked well on Two-Room and Reacher, and failed on the benchmark environments requiring an understanding of contact dynamics and 3D structure. This chapter places these findings in the context of the wider literature, then covers the main limitations, what should have been done differently, and future work.

6.1 Interpretation and Comparison with World Modelling Literature

The limitations were failures of what the world model represented and how faithfully it modelled dynamics. Push-T ignored the pusher or drifted from the real environment, and OGBench-Cube failed to track height. Planning can not be done using information that is not there. This aligns with conclusions made in the latent-model literature. Rich spatial features are necessary for fine-grained manipulation. DINO-WM reports a large Push-T gap between spatial patch features and a global image embedding [63], and linear probes on LeWM’s embeddings recover agent and block pose despite the model never being given them explicitly [34]. The trade-off is representational density. Latent models, pretrained at scale, retain information they were never asked to name; explicit code, written from a handful of trajectories, keeps only what the VLM chose to write down [46, 14]. When success depends on small differences in how objects interact, leaving information out undermines the benefits of having explicit, readable code. Annotating every task-relevant entity in natural language would make omissions like the missing pusher less likely, but supplying the generator with a checklist of what matters is closer to captioning the scene than inferring dynamics visually,

defeating the purpose of allowing a VLM to generate a model only from visual-action information. For these tasks, more visual examples may help improve performance. For example, for OGBench-cube, being able to observe a scene from different viewpoints may give the VLM more information about the dynamics.

The VLM repeatedly diagnosed its own defects and shipped them anyway. Traces on both Two-Room and Reacher named the exact fault. These were walls being ignored or damping set too low, yet both survived into the frozen code. In the literature, VLMs usually have trouble locating the error at all [24, 52]. Here localisation was clear and the repair should have been a trivial task, but it was not fixed. What was missing may have been the reason to fix it. For example, in Two-Room, the expert validation demonstration never collided with a wall, so every internal check passed, even though the code had a defect that made it fail on harder examples during P2. Why the VLM identified a defect and then shipped it unrepaired would need its own study to answer.

The offline internal gate-validator was another problem. This offline gate did not establish reliability in these experiments. It may be that no purely offline validation can suffice, and that limited real-world interaction is required to correct transitions the VLM has misspecified. Chua et al [11] frame the risk in terms of the planner acting as an optimiser, gravitating toward regions of high predicted reward that may be overconfident artefacts of model noise. Dainese et al. [14] demonstrate this same gap, showing that high offline predictive accuracy does not translate to effective decision-making when the model misrepresents transition logic necessary in the real environment. Incorporating some access to the actual environment, through its own interactions, may therefore be necessary for a VLM to create a truly faithful world model.

Varied demonstrations did not help in the way the literature suggested. PLDM finds that diverse data can improve latent planning through broader coverage [42]. Here, mixed compositions changed how simulators read the goal but helped neither Push-T nor Cube. Coverage helps a latent model, which absorbs everything it sees, but not a VLM, which does not always name what matters. As a result, varied demonstrations were not linked to better model performance in this study.

This approach’s advantage over latent models lies in interpretable, code-level failure modes, since the defects here were clear in a few lines of code, which is not true of a latent embedding. However, readable rules are not enough if the VLM fails to track what the world model needs to be faithful, and what planning requires. The clearest practical advantage of this system is data efficiency. Two-Room and Reacher succeeded from a few example trajectories rather than the roughly 10,000 episodes used by the latent models [63, 34], and no task-specific weights were trained at all.

6.2 Critical Reflection on Methodology

The project developed a single, generalised system that was sequentially applied to four benchmarks, scaling in complexity from 2D Two-Room to 3D OGBench-Cube. This was the wrong order. Engineering the system to work on each benchmark environment introduced changes that interfered with earlier ones, and recovering from degraded performance on simple benchmarks became a trial-and-error loop. A better approach would have been to build a deliberately over-fitted system for each benchmark environment first, establish what a working pipeline needs in each case, and only then strip away what proved unnecessary to find the general system. Generalisation would then mean combining four known-good solutions, rather than engineering one system to suit all four environments at once.

The 2D/3D conflation was the main instance of this problem. Flat-image and volumetric tasks need different sets of variables, different perception and different validation, and forcing both through one interface produced simulators that defaulted to the 2D case even where height decided the outcome. The available Gemini-2.5 perception may also have struggled on these synthetic renders, pushing the VLM further towards flat representations. The planar and volumetric tasks should have been developed as separate systems and combined only once each worked.

The tooling had limits too. Even with the generated code and the reasoning traces available, it was often unclear during experimentation why a prompt change failed to fix a defect. Working on one benchmark in isolation would have made that far more tractable than diagnosing it inside a system also trying to do navigation. Code inspection localises faults well, but it does not by itself explain why the VLM keeps producing them.

There are three limits to how the results can be interpreted. The first one is that the VLM is one model, so the manipulation failures may be a limit of Gemini 3.6 Flash’s capabilities, and nothing here assesses that possibility. The second is that the planner is also one implementation, and different horizons were not assessed (Section 3.7). With $H = K = 25$, each plan ran open-loop for half the episode, giving contact-model error a long window to accumulate. Replanning more often might have limited it, so the Push-T result is partly a statement about the planner settings being poorly suited rather than the world model alone. Finally, the design can not say where the dynamics came from. A VLM trained on physics text may be making up dynamics based on its pre-training data rather than inferring dynamics from the eight trajectories. No experiment here tested that.

6.3 Future Work

1. Focus on 2D and 3D dynamics separately. Push-T needs every moving entity present in the state, target and cost, with contact parameters explicitly fitted rather than guessed, and this is best developed against Push-T alone, not inside a general system. OGBench-Cube needs its own system that focuses on 3D dynamics understanding. This may be helped by using perception from depth, stereo vision [57], or a 3D reconstruction tool [50].

2. Include more 3D information. Incorporate different points of view for 3D environments as this may help the VLM understand the scene better.

3. Repair code defects by hand to see if performance improves. Each defect, including the missing collision check, the weak damping, the absent pusher term, should be edited in isolation by a human and the evaluation rerun. This would close the loop from a visible error to a measured improvement. Until it is done, the claim that readable code makes debugging easier remains an argument rather than a demonstration.

4. Experiment with stronger VLMs. The pipeline treats the VLM as a swappable component, so a stronger VLM could be substituted without altering the pipeline, the tooling or the evaluation. Whether the manipulation failures reflect a limit of this VLM or of the approach itself is an open question that a direct model comparison would settle.

Chapter 7

Conclusion

This dissertation asked whether a VLM could turn a handful of RGB–action trajectories into world models a planner can use and a human can read. It can, within the limits the results show.

The novelty is not the performance but the architecture. Where a latent world model absorbs a scene into an embedding and offers no insight into what it got wrong, this pipeline commits its understanding to Python code. It makes the system’s blind spots visible, its failures locatable, and its improvement incremental rather than a matter of retraining.

These are the answers to the three research questions.

RQ1 Every attempt produced executable code, while only a minority passed internal validation. The system generates functional simulators for simple tasks but struggles with contact dynamics and 3D scenes. The validation gate was insufficient to guarantee the generated simulator’s fidelity to the real environment.

RQ2 While the generated simulators matched or outperformed LeWM on Two-Room and Reacher, they failed on tasks requiring contact dynamics or metric height, where LeWM had a substantial advantage. The internal validation gates did not reliably predict test-time control, so passing them says little about whether a world model is any good.

RQ3 Neither the demonstration mixture nor repeated generation helped the manipulation tasks, and runs from an identical configuration varied widely, so a single generation is not a reliable sample of what the pipeline produces. Code inspection, however, localised a specific missing quantity on every benchmark.

The failures are the most valuable information. Each resolved to a specific line of code, be that a quantity left unrepresented, or one parametrised wrongly, pointing directly at the

function that would have to change. A latent model offers no such information. That is the difference between a world model that can be debugged and one that can only be retrained.

Manipulation remains unsolved, and the pipeline must be revised. A system that states its own terms of failure, however, can be improved deliberately. If robots are to build models of the world they act in, there is value in those models being ones a person can read, argue with and correct, rather than ones that can only be retrained and trusted. That is the case this dissertation makes for explicit action-conditioned visual world models, and the results mark both its promise and where it needs refinement.

References

- [1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a Joint-Embedding predictive architecture, 2023. URL <https://arxiv.org/abs/2301.08243>. arXiv:2301.08243 [cs.CV].
- [2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning, 2025. URL <https://arxiv.org/abs/2506.09985>. arXiv:2506.09985 [cs.AI].
- [3] Ashay Athalye, Nishanth Kumar, Tom Silver, Yichao Liang, Jiuguang Wang, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. From pixels to predicates: Learning symbolic world models via pretrained Vision-Language models, 2026. URL <https://arxiv.org/abs/2501.00296>. arXiv:2501.00296 [cs.RO].
- [4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. URL <https://arxiv.org/abs/2404.08471>. arXiv:2404.08471 [cs.CV].
- [5] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A Vision-Language-Action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>. arXiv:2410.24164 [cs.LG].
- [6] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu,

- Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-2: Vision-Language-Action models transfer web knowledge to robotic control, 2023. URL <https://arxiv.org/abs/2307.15818>. arXiv:2307.15818 [cs.RO].
- [7] Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments, 2024. URL <https://arxiv.org/abs/2402.15391>. arXiv:2402.15391 [cs.LG].
- [8] Maxime Burchi and Radu Timofte. MuDreamer: Learning predictive world models without reconstruction, 2024. URL <https://arxiv.org/abs/2405.15083>. arXiv:2405.15083 [cs.AI].
- [9] Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, Mustafa Shukor, Jess Moss, Alexander Soare, Dana Aubakirova, Quentin Lhoest, Quentin Gallouédec, and Thomas Wolf. LeRobot: An open-source library for End-to-End robot learning, 2026. URL <https://arxiv.org/abs/2602.22818>. arXiv:2602.22818 [cs.RO].
- [10] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion, 2024. URL <https://arxiv.org/abs/2303.04137>. arXiv:2303.04137 [cs.RO].
- [11] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models, 2018. URL <https://arxiv.org/abs/1805.12114>. arXiv:1805.12114 [cs.LG].
- [12] Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. Technical report, Google, 2025. URL <https://arxiv.org/abs/2507.06261>. arXiv:2507.06261 [cs.CL].
- [13] Kenneth James Williams Craik. *The Nature of Explanation*. Cambridge University Press, Cambridge, UK, 1943.
- [14] Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search, 2024. URL <https://arxiv.org/abs/2405.15383>. arXiv:2405.15383 [cs.AI].
- [15] Pieter-Tjerk de Boer, Dirk Kroese, Shie Mannor, and Reuven Rubinstein. A tutorial on the Cross-Entropy method. *Annals of Operations Research*, 134(1):19–67, February 2005. doi: 10.1007/s10479-005-5724-z. URL <https://ideas.repec.org/a/spr/annopr/v134y2005i1p19-6710.1007-s10479-005-5724-z.html>.
- [16] Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion, 2017. URL <https://arxiv.org/abs/1610.00696>. arXiv:1610.00696 [cs.LG].

- [17] Google. Gemini 3.6 Flash large language model. Software model, 2026. URL <https://deepmind.google/models/gemini/flash/>. <https://deepmind.google/models/gemini/flash/>. Accessed 3 August 2026.
- [18] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training, 2022. URL <https://arxiv.org/abs/2211.11559>. arXiv:2211.11559 [cs.CV].
- [19] David Ha and Jürgen Schmidhuber. World models, 2018. URL <https://arxiv.org/abs/1803.10122>. arXiv:1803.10122 [cs.LG].
- [20] Sami Haddadin. The Franka Emika robot: A standard platform in robotics research [survey]. *IEEE Robotics & Automation Magazine*, 31(4):136–148, 2024. doi: 10.1109/MRA.2024.3451788.
- [21] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels, 2019. URL <https://arxiv.org/abs/1811.04551>. arXiv:1811.04551 [cs.LG].
- [22] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination, 2020. URL <https://arxiv.org/abs/1912.01603>. arXiv:1912.01603 [cs.LG].
- [23] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control, 2024. URL <https://arxiv.org/abs/2310.16828>. arXiv:2310.16828 [cs.LG].
- [24] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet, 2024. URL <https://arxiv.org/abs/2310.01798>. arXiv:2310.01798 [cs.CL].
- [25] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation, 2026. URL <https://arxiv.org/abs/2601.03782>. arXiv:2601.03782 [cs.RO].
- [26] Nabil Kahouadji. A comprehensive comparison of the Wald, Wilson, and adjusted Wilson confidence intervals for proportions, 2025. URL <https://arxiv.org/abs/2508.10223>. arXiv:2508.10223 [stat.ME].
- [27] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source Vision-Language-Action model, 2024. URL <https://arxiv.org/abs/2406.09246>. arXiv:2406.09246 [cs.RO].
- [28] Yann LeCun. A path towards autonomous machine intelligence, version 0.9.2. Technical report, Courant Institute of Mathematical Sciences, New York University, jun 2022. URL <https://openreview.net/pdf?id=BZ5a1r-kVsf>. <https://openreview.net/pdf?id=BZ5a1r-kVsf>.

- [29] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-End training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016. URL <https://dl.acm.org/doi/10.5555/2946645.2946684>.
- [30] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control, 2023. URL <https://arxiv.org/abs/2209.07753>. arXiv:2209.07753 [cs.RO].
- [31] Haowen Liu, Shaoxiong Yao, Haonan Chen, Jiawei Gao, Jiayuan Mao, Jia-Bin Huang, and Yilun Du. SIMPACT: Simulation-enabled action planning using Vision-Language models, 2026. URL <https://arxiv.org/abs/2512.05955>. arXiv:2512.05955 [cs.RO].
- [32] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models, 2024. URL <https://arxiv.org/abs/2310.12931>. arXiv:2310.12931 [cs.RO].
- [33] Lucas Maes, Quentin Le Lidec, Dan Haramati, Nassim Massaudi, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel-v1: Reproducible world modeling research and evaluation, 2026. URL <https://arxiv.org/abs/2602.08968>. arXiv:2602.08968 [cs.AI].
- [34] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable End-to-End Joint-Embedding predictive architecture from pixels, 2026. URL <https://arxiv.org/abs/2603.19312>. arXiv:2603.19312 [cs.LG].
- [35] D.Q. Mayne, J.B. Rawlings, C.V. Rao, and P.O.M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000. ISSN 0005-1098. doi: [https://doi.org/10.1016/S0005-1098\(99\)00214-9](https://doi.org/10.1016/S0005-1098(99)00214-9). URL <https://www.sciencedirect.com/science/article/pii/S0005109899002149>.
- [36] Simon McIntosh-Smith, Sadaf Alam, and Christopher Woods. Isambard-AI: A leadership-class supercomputer optimised specifically for artificial intelligence. In *Proceedings of the Cray User Group*, CUG '24, pages 44–54, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400713286. doi: 10.1145/3725789.3725794. URL <https://doi.org/10.1145/3725789.3725794>.
- [37] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- [38] Heejeong Nam, Quentin Le Lidec, Lucas Maes, Yann LeCun, and Randall Balestriero. Causal-JEPA: Learning world models through object-level latent masking, 2026. URL <https://arxiv.org/abs/2602.11389>. arXiv:2602.11389 [cs.AI].
- [39] Felix O’Mahony, Roberto Cipolla, and Ayush Tewari. VDAWorld: World modelling via VLM-Directed abstraction and simulation, 2026. URL <https://arxiv.org/abs/2512.11061>. arXiv:2512.11061 [cs.CV].
- [40] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL, 2025. URL <https://arxiv.org/abs/2410.20092>. arXiv:2410.20092 [cs.LG].

- [41] Wasu Top Piriyaikulij, Yichao Liang, Hao Tang, Adrian Weller, Marta Kryven, and Kevin Ellis. PoE-World: Compositional world modeling with products of programmatic experts, 2025. URL <https://arxiv.org/abs/2505.10819>. arXiv:2505.10819 [cs.AI].
- [42] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestrieri, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models, 2025. URL <https://arxiv.org/abs/2502.14819>. arXiv:2502.14819 [cs.LG].
- [43] Jonathan Spieler, Angel Villar-Corrales, and Sven Behnke. Slot-MPC: Goal-conditioned model predictive control with object-centric representations, 2026. URL <https://arxiv.org/abs/2605.14937>. arXiv:2605.14937 [cs.LG].
- [44] Dídac Surís, Sachit Menon, and Carl Vondrick. ViperGPT: Visual inference via Python execution for reasoning, 2023. URL <https://arxiv.org/abs/2303.08128>. arXiv:2303.08128 [cs.CV].
- [45] Richard S Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Machine learning proceedings 1990*, pages 216–224. Elsevier, 1990.
- [46] Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment, 2024. URL <https://arxiv.org/abs/2402.12275>. arXiv:2402.12275 [cs.AI].
- [47] A. Tasora, R. Serban, H. Mazhar, A. Pazouki, D. Melanz, J. Fleischmann, M. Taylor, H. Sugiyama, and D. Negrut. Chrono: An open source multi-physics dynamics engine. In T. Kozubek, editor, *High Performance Computing in Science and Engineering*, volume 9611 of *Lecture Notes in Computer Science*, pages 19–49. Springer, 2016.
- [48] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. DeepMind control suite, 2018. URL <https://arxiv.org/abs/1801.00690>. arXiv:1801.00690 [cs.AI].
- [49] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy, 2024. URL <https://arxiv.org/abs/2405.12213>. arXiv:2405.12213 [cs.RO].
- [50] SAM 3D Team, Xingyu Chen, Fu-Jen Chu, Pierre Gleize, Kevin J Liang, Alexander Sax, Hao Tang, Weiyao Wang, Michelle Guo, Thibaut Hardin, Xiang Li, Aohan Lin, Jiawei Liu, Ziqi Ma, Anushka Sagar, Bowen Song, Xiaodong Wang, Jianing Yang, Bowen Zhang, Piotr Dollár, Georgia Gkioxari, Matt Feiszli, and Jitendra Malik. SAM 3D: 3Dfy anything in images, 2026. URL <https://arxiv.org/abs/2511.16624>. arXiv:2511.16624 [cs.CV].
- [51] Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with Joint-Embedding predictive world models?, 2026. URL <https://arxiv.org/abs/2512.24497>. arXiv:2512.24497 [cs.AI].

- [52] Gladys Tyen, Hassan Mansoor, Victor Cărbune, Peter Chen, and Tony Mak. LLMs cannot find reasoning errors, but can correct them given the error location, 2024. URL <https://arxiv.org/abs/2311.08516>. arXiv:2311.08516 [cs.AI].
- [53] Hongyu Wang, Jingquan Wang, Bocheng Zou, Radu Serban, and Dan Negrut. Coding agent is good as world simulator, 2026. URL <https://arxiv.org/abs/2605.14398>. arXiv:2605.14398 [cs.AI].
- [54] Lirui Wang, Yiyang Ling, Zhecheng Yuan, Mohit Shridhar, Chen Bao, Yuzhe Qin, Bailin Wang, Huazhe Xu, and Xiaolong Wang. GenSim: Generating robotic simulation tasks via large language models, 2023. URL <https://arxiv.org/abs/2310.01361>. arXiv:2310.01361 [cs.RO].
- [55] Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning, 2026. URL <https://arxiv.org/abs/2603.12231>. arXiv:2603.12231 [cs.LG].
- [56] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation, 2024. URL <https://arxiv.org/abs/2311.01455>. arXiv:2311.01455 [cs.RO].
- [57] Bowen Wen, Matthew Trepte, Joseph Aribido, Jan Kautz, Orazio Gallo, and Stan Birchfield. FoundationStereo: Zero-shot stereo matching, 2025. URL <https://arxiv.org/abs/2501.09898>. arXiv:2501.09898 [cs.CV].
- [58] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Ken Goldberg, and Pieter Abbeel. DayDreamer: World models for physical robot learning, 2022. URL <https://arxiv.org/abs/2206.14176>. arXiv:2206.14176 [cs.RO].
- [59] Sherry Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Leslie Kaelbling, Dale Schuurmans, and Pieter Abbeel. Learning interactive Real-World simulators, 2024. URL <https://arxiv.org/abs/2310.06114>. arXiv:2310.06114 [cs.AI].
- [60] Shicheng Yin, Kaixuan Yin, Weixing Chen, Yang Liu, Guanbin Li, and Liang Lin. DDP-WM: Disentangled dynamics prediction for efficient world models, 2026. URL <https://arxiv.org/abs/2602.01780>. arXiv:2602.01780 [cs.CV].
- [61] Jerrold H. Zar. Spearman rank correlation: Overview. In *Wiley StatsRef: Statistics Reference Online*. John Wiley & Sons, Ltd, 2014. ISBN 9781118445112. doi: <https://doi.org/10.1002/9781118445112.stat05964>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781118445112.stat05964>.
- [62] Kai Zhao, Dongliang Nie, Yuchen Lin, Zhehan Luo, Yixiao Gu, Deng-Ping Fan, and Dan Zeng. Sub-JEPA: Subspace Gaussian regularization for stable End-to-End world models, 2026. URL <https://arxiv.org/abs/2605.09241>. arXiv:2605.09241 [cs.LG].
- [63] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning, 2025. URL <https://arxiv.org/abs/2411.04983>. arXiv:2411.04983 [cs.RO].